Management and Analysis of Physics Datasets - Mod. B

Gabriele Bellavia

October 12, 2026

About these notes

These notes are part of my `PoD-collection-of-notes` repository and were written during my MSc in Physics of Data at the University of Padua between 2024 and 2026.

They are intended as freely accessible study material and may contain errors, inaccuracies, or outdated information. Most of the material was written while attending the courses and was not systematically reviewed afterwards. Course contents and syllabi may also change over time.

The first six chapters of these notes are basically an english translation, with some modifications, of lecture notes by Dario Liotta, available at

`https://github.com/darioliotta`

The remaining chapters are based on my own lecture notes and course material.

License

Except where otherwise noted, this work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0).

You are free to share and adapt this material for non-commercial purposes, provided that appropriate credit is given, a reference to the original source is included, and any derivative work is distributed under the same license.

Source files

The complete source files used to compile these notes, including figures and other resources, are available in the corresponding folder of the repository:

`https://github.com/gabriele-bellavia/PoD-collection-of-notes`

Citation

If you use these notes, redistribute them, or use them as a starting point for your own material, please cite the original repository and, where applicable, the authors of any external material referenced in these notes.

Errors and corrections

If you find an error, typo, or unclear explanation, please feel free to open an issue on the GitHub repository or contact me at:

`gabriele.bellavia@studenti.unipd.it`

Physics of Data MSc
University of Padua

Contents

1 Data Management **1**
- 1.1 Introduction: from sensors to data . . . 1
- 1.2 From data to dataset . . . 2
  - 1.2.1 Metadata . . . 3
  - 1.2.2 Data structure . . . 3

2 Storage and Scalability **5**
- 2.1 Data Storage . . . 5
  - 2.1.1 Storage media . . . 6
  - 2.1.2 Kryder's law . . . 8
- 2.2 Scalability . . . 9

3 Reliability and Security **11**
- 3.1 Reliability . . . 11
  - 3.1.1 Reliability strategies . . . 12
- 3.2 Security . . . 16
  - 3.2.1 Cryptography . . . 16
  - 3.2.2 Authentication and authorization . . . 17
  - 3.2.3 Encoding/decoding information . . . 17
  - 3.2.4 Hash functions . . . 18
  - 3.2.5 Encryption vs hashing algorithms . . . 20

4 File System **21**
- 4.1 Characteristics . . . 21
- 4.2 Limits . . . 25
- 4.3 From local to distributed file system . . . 26
  - 4.3.1 Functioning . . . 27
  - 4.3.2 A DFS example: HDFS . . . 28
  - 4.3.3 CAP Theorem . . . 31

5 Databases **35**
- 5.1 Databases models . . . 36
- 5.2 Relational databases . . . 37
  - 5.2.1 SQL . . . 40

6 NoSQL **52**
- 6.1 NoSQL databases . . . 52
- 6.2 Databases partition . . . 52

6.3 Characteristics of NoSQL databases . . . 54
6.4 NoSQL databases examples . . . 55

7 Intro to Data Processing **57**
7.1 Processing . . . 57
7.1.1 Processing time . . . 58
7.1.2 Power wall . . . 58
7.1.3 Pipelining . . . 59
7.2 Multi-processing architectures . . . 59
7.3 Parallelism . . . 59
7.3.1 Parallel processing . . . 60
7.3.2 Flynn's taxonomy . . . 60
7.3.3 Speedup in parallel programming . . . 62
7.3.4 Amdhal's law . . . 63
7.3.5 Gustafson's law . . . 63

8 Intro to Concurrency **64**
8.1 Processes and threads . . . 64
8.2 Concurrency and parallelism . . . 65
8.2.1 Multi-tasking . . . 65
8.2.2 Multi-threading . . . 66
8.2.3 Race conditions . . . 68
8.2.4 Thread synchronization . . . 68
8.3 GIL: Global Interpreter Lock . . . 69
8.4 Summary: in the end threads or processes in python? . . . 69

9 Intro to Distributed Processing **71**
9.1 Parallel programming . . . 71
9.2 Parallel computing architectures . . . 71
9.3 Distributed computing: cluster . . . 72
9.4 A common job submission pattern: batch systems . . . 73

10 Map-Reduce and Hadoop **75**
10.1 Big data . . . 75
10.2 Hadoop . . . 76
10.2.1 HDFS recap . . . 77
10.2.2 Map-Reduce in Hadoop . . . 77
10.2.3 YARN . . . 79
10.2.4 Hadoop high-level abstractions . . . 80

11 Spark **81**
11.1 Spark vs Hadoop . . . 81
11.2 Spark cluster architecture . . . 82
11.2.1 RDDs . . . 83
11.2.2 Transformations and actions . . . 84
11.2.3 DAG scheduler . . . 85
11.2.4 DataFrame . . . 87
11.3 Real time analytics . . . 87
11.3.1 Real time analytics in Spark . . . 88

11.3.2 Real time analytics in Spark - RDD 89
11.3.3 Real time analytics in Spark - DataFrame 90

12 Dask **91**
12.1 Dask introduction 91
12.2 High-level API 92
12.3 Low-level API 92
12.4 DAG scheduler 93
12.5 Dask workflow 94
12.6 Dask vs Spark 95

Chapter 1

Data Management

1.1 Introduction: from sensors to data

In general, the process that transforms sensor signals into usable data is more complex than it may seem. Specifically, the chain from sensors to triggers and data acquisition systems (DAQ) only produces **raw data**, not actual *information*. This entire chain consists of

- **Sensors** → converts physical quantities into electrical signals.
- **Front-end electronic** → serve as the interface between sensor and the signal processing system. The signal is amplified and filtered to remove undesired noise.
- **Read-out electronics** → are responsible for converting analog signal into digital ones.
- **Trigger and DAQ** → the trigger determines the exact moment when the system starts to acquire data, e.g. when an input signal exceeds a defined threshold. The DAQ system filters the signal and store it.

This is collectively referred to as the **DAQ system**.

After data is acquired, the **computing model** comes into play. This involves

- **Storage** → method and location where the data obtained from DAQ is physically stored.
- **Reliability and security** → techniques used to preserve data integrity and to control access permissions.
- **Querying and filtering** → the process of searching for and selecting data. Querying involves extracting data from memory-based and predefined criteria, allowing users to retrieve only the relevant information.
- **Processing** → involves the use of algorithms and computational tools to extract useful information from stored data.

Only after all these steps we can extract *meaningful information* from raw data. This can all be summarized in Figure 1.1

Clearly, the sources may be different:

- from sensors: experiments/IoT/etc...

- simulated data: montecarlo/...
- text: log files/documents/etc...
- heterogeneous sources: GPS location + image + ...

To have an idea, we have to ask ourselves how a computing model scales? The answer is: the amount of raw data; the need of securing and preserving the data for a long period of time; the number of users accessing the data at the same time; the complexity of the processing.

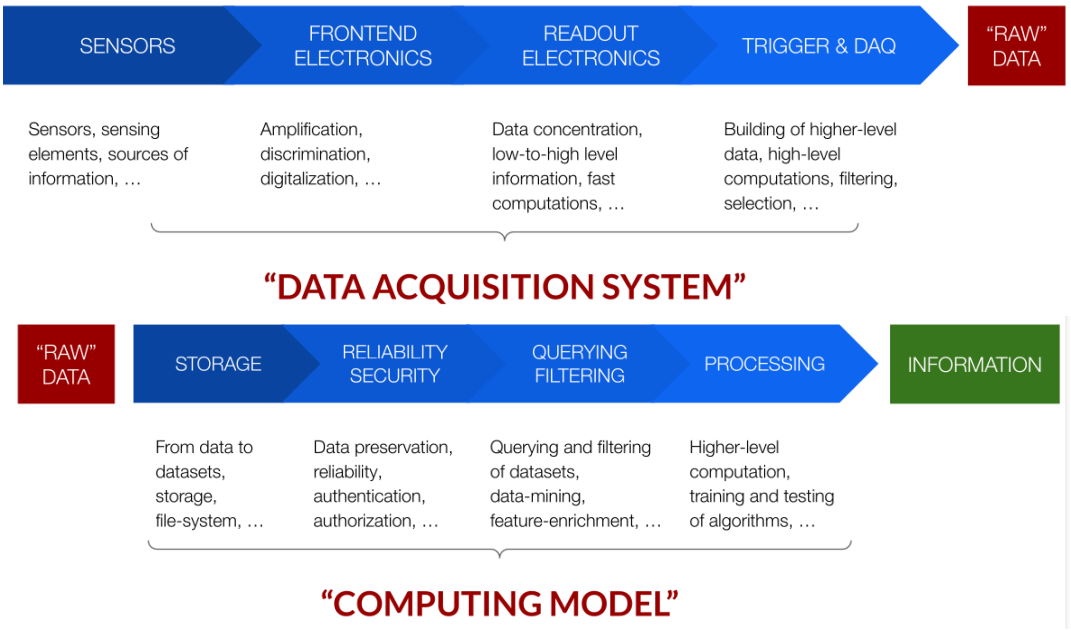


Figure 1.1

1.2 From data to dataset

A **dataset** is a collection of information – whether from real data or simulations – gathered from various sources (such as detectors or other devices) over a specific period of time and associated with a common goal or context. Every single *datum*, that is a "piece of data", is often referred to as a **record**.

A dataset is constructed in the following way (illustrated in Figure 1.2):

1. A DAQ writes continuously in memory at high-rate every information taken from sensors.
2. Stored data typically undergoes an initial processing phase, usually a few hours or days after acquisition. This includes data cleaning, error correction, normalization and, if necessary, dimensionality reduction.
3. On average, data undergoes further processing once a year, involving more sophisticated techniques, such as MC simulations.
4. Finally, processed data is stored long-term in a dedicated memory space.

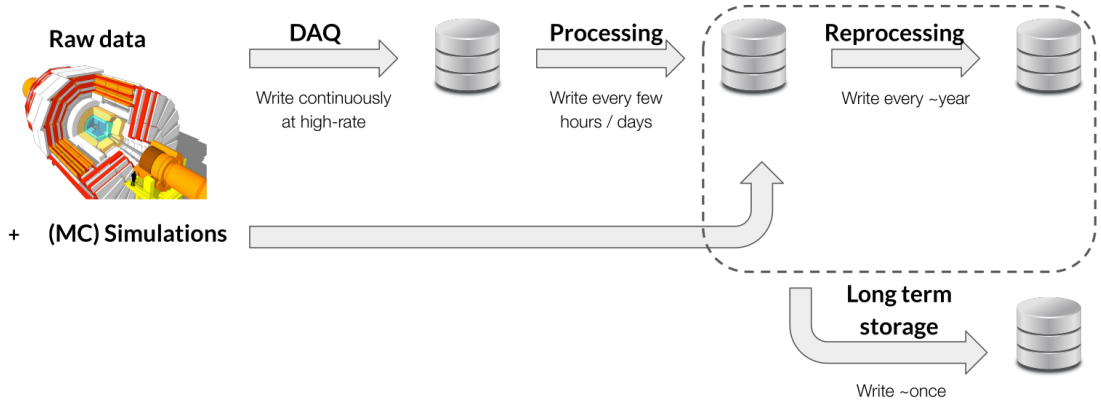


Figure 1.2

1.2.1 Metadata

Data should always be accompanied by **metadata**. These are "*data about data*". Metadata capture information describing the dataset itself, providing valuable context to help interpret the data and enabling its correct access, description and processing.

Metadata serve three main functions:

- **Descriptive**: what does if represent? Who is the owner? What does it contain? When was it published?
- **Structural**: how it is connected to other datasets? How it is organized? According to what criteria has it been divided? Is it sorted?
- **Administrative**: when was it created, modified or archived? Who can access it? How large is it?

It is clear that without metadata, it becomes impossible to understand an "abstracted" dataset, such as one produced by a particle physics experiment, where the raw data alone carries little meaning without contextual information.

1.2.2 Data structure

Datasets can be classified into three main types based on their structure and organization:

- **Structured data**: follows a rigid tabular structure, with each data point located at a unique intersection of a row and a column. For example, raw data are not readable because they are written in hexadecimal form. A `CSV` file or `DBMS` is structured data in tabular form. The main advantage is the easy readability. The cons are due to the rigidity of this schema, that is the limitation in adding new columns or the limited flexibility to correct some errors.
- **Semi-structured data**: is organized according to a structured logic, but it does not follow a fixed schema defined in advance. Instead, the structure can be inferred from the data itself through the use of **tags** and **markup** elements. For example, semi-structured data files are `XML`, `JSON` and `YAML`. In order to verify integrity of data, it is always possible to convert semi-structured data into structured ones.

- **Unstructured data**: is a type of data with no predefined structure. There is no predefined data schema, nor it can be inferred from data itself. These datasets are typically text-heavy, but may also include audio, video or images, geolocation data, timestamps, numeric values.

  Unstructured data is typically generated as a continuous flow of information, such as natural language text, audio recordings, video streams, image sequences.

  The main advantage of this format is its flexibility: it can adapt to any type of data. However, a major drawback is the difficulty of extracting specific pieces of information, due to the lack of structure.

Chapter 2

Storage and Scalability

2.1 Data Storage

We saw that each storage unit plays a different role (Figure 1.2), even though the overall goal is to retain data in memory:

- The DAQ writes into memory at high frequency using multiple writing cycles in a short amount of time. However, these data will not remain in that memory unit for long: they are quickly processed or transferred elsewhere, freeing up space to be filled with new incoming information.
- The processed data are distributed into two different units: in both cases writing process requires longer time spans. At the same time, the required quantity of memory grows a lot.
- The same data are copied in another memory unit which does not require reading.

Hence, we can observe different needs depending on the type of memory unit. A decreasing number of read/write operations corresponds to an increasing amount of available storage space. These different requirements translate into different technologies designed to meet them. These technologies differ in several aspects related to input/output requirements, such as reading, writing, updating and deleting data. The aspects are the following:

- **I/O**: request for input/output operations (read, write, update, ...).
- **Latency**[s]: time taken to complete the I/O request.
- **Rate**[Hz]: number of I/O that can be performed in a second of a given workload (**IOPS** $\rightarrow$ I/O operations per second).
- **Block size**[byte]: average size of a I/O request of a given workload.
- **Bandwidth**[bytes/s]: theoretical capacity of data transfer over time.
- **Throughput**[bytes/s]: actual capacity of data transfer over time.

Example: write 1000 blocks of 4 kB each. The I/O operation is a write operation and the block size is 4 kB. Hence, the total transfer size is

$$\text{size} = 1000 \cdot 4\,\text{kB} = 4\,\text{MB}$$

The measured latency is 11.1 ms, then the estimated throughput is

$$\text{E.T.} = \frac{\text{tot. transfer size}}{\text{measured latency}} = 360\,\text{MB/s}$$

and the estimated IOPS is

$$\text{E. IOPS} = \frac{\text{\# transfers}}{\text{measured latency}} = 9000\,\text{IOPS}$$

2.1.1 Storage media

The following media are listed from the lowest to the highest latency.

Dynamic Random Access Memory (DRAM)

DRAMs store data bits in `NAND` cells, using semiconductor memory technology. As the name RAM suggests, data can be accessed from every position at the same speed. The term *dynamic* indicates that the data is not retained permanently: it is lost when power is turned off. This is the main drawback of DRAM. It is a temporary storage medium.

However, its main advantage is the high speed of I/O operations, with a latency of about 0.1 μs. For this reason, DRAM is typically used in processes that require a high frequency of write operations.

Non-Volatile Memory (NVM)

This is an alternative to DRAM, offering a similar latency of around 1 μs and using several different number of technologies. The main advantage is that NVM retains data even if the power is off. However, NVM is a highly specialized storage solution, and it is rarely used in standard applications due to its high cost.

Solid State Drives (SSD)

SSDs are widely used today due to their reduced cost and the absence of mechanical components – similar to DRAMs and NVMs.

Most SSDs are based on a `NAND` flash technology, using multi-level charge traps to store data in a compact and high density format. Typically, 8 levels are used to encode 3 bits of information, a structure called triple level cell (TLC).

SSDs are especially useful in applications that require large storage capacity and low latency, with read latencies of about 10 μs and write operations of 100 μ. This slow write latency is due to a physical process in which, in order to write in the cell, the entire content must first be erased and the capacitor discharged. This leads to another limitation caused by the hysteresis effect: each write cycle leaves behind a small residual charge in the cell. Over time, this accumulation compromises the cell's functionality.

To manage this, SSDs include a controller that tracks the number of writes per cell. Once a defined threshold is exceeded, the cell is marked as unusable. Consequently, SSDs have a limited lifespan and an effective expiration date. While they are perfectly suited for everyday consumer use, they may not be appropriate for research environments that involve high-frequency write operations or require long-term reliability.

Hard Disk Drive (HDD)

HDDs are still the most widely used storage devices today, mainly due to their low cost and relatively high performance for many standard applications. However, they are primarily suitable for medium-to-high latency applications.

An HDD is composed of a set of double-sided spinning platters, each coated with a magnetic medium. Data is stored by flipping the magnetic field across two adjacent domains, rather than associating a bit to a single track. Specifically, a 0 bit corresponds to two tracks with the same polarization, while a 1 bit corresponds to opposite polarization. These domains are arranged in concentric circles called tracks, which are grouped into sectors, typically ranging from 0.5 to 4 kB in size. These sectors represent the block size used during read and write operations.

Data I/O is handled by a moving arm assembly that positions the read/write head over the appropriate track to access the desired sector. Hence, data is accessed sector by sector. The rotation of the platters allows alignment with the correct sector, while the mechanical arm ensures alignment with the correct track. Their performance is limited by their mechanical components, resulting in latencies in the range of ms, which can be broken into two components:

- seek time: the time needed to align the head with the correct track
- rotational delay: time needed for the platter to rotate the correct sector under the head.

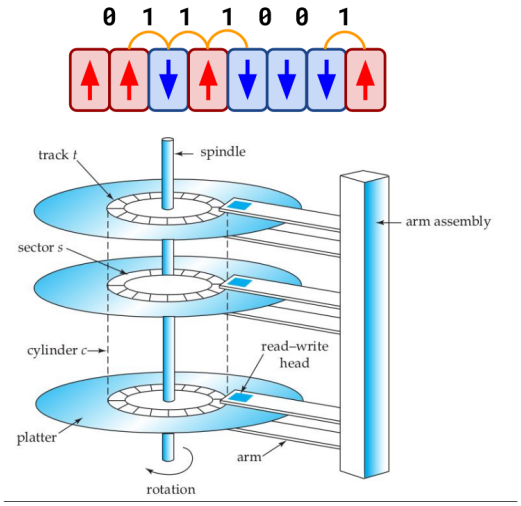


Figure 2.1

Magnetic tape

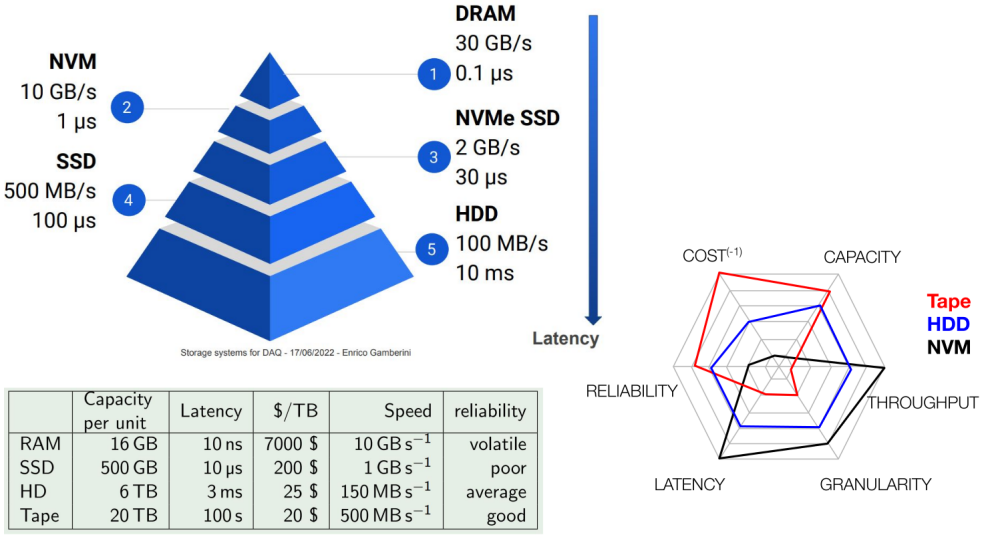
Magnetic tape functions similarly to HDDs in their use of magnetic domains for data storage, but differs in that these domains are arranged sequentially along a flexible tape medium, which can be rolled up into compact cartridges. These cartridges are typically managed by robotic arms in large-scale storage facilities, which handle their movement and swapping as needed.

This storage format is mainly used for archival purposes, where data is accessed infrequently or after long time intervals. When a data access request is made, several steps must occur:

1. A robotic arm fetches the appropriate cartridge
2. The tape is unrolled to the point where the requested data begins.
3. The system reads the data sequentially along the storage locations.

Although the read operation itself is fast, the overall latency of the process is relatively high – typically in the order of 100 s or more – due to mechanical steps involved and possible delays from other concurrent operations. Moreover, since entire files are written and read sequentially, accessing random data is not efficient.

Despite this, magnetic tapes offer two major advantages: their very low production costs and their remarkable durability. Thanks to their physical resilience, data stored on tapes can survive extreme conditions – such as floods or mechanical shocks – without being lost, making them an excellent solution for long-term, low-cost data preservation.



	Capacity per unit	Latency	$/TB	Speed	reliability
RAM	16 GB	10 ns	7000 $	10 GB s^{-1}	volatile
SSD	500 GB	10 µs	200 $	1 GB s^{-1}	poor
HD	6 TB	3 ms	25 $	150 MB s^{-1}	average
Tape	20 TB	100 s	20 $	500 MB s^{-1}	good

Figure 2.2: Storage systems properties.

2.1.2 Kryder's law

In the context of data storage, the **Kryder's law** is considered the analogue of the more famous Moore's law, which states that the number of transistors in an integrated circuit doubles approximately every year. Similarly, Kryder's law – proposed in 1998 based on empirical observations – states that the storage density would double every 13-14 months.

However, this trend has not held true since around 2010, due to physical limitations in current storage media. Despite that, the underlying message is that the field of data storage technologies evolves very rapidly, and this must be taken into account when managing datasets that grow continuously over years.

2.2 Scalability

As most datasets are expected to grow over time – whether generated by ongoing experiments, industrial processes or users' produced content – a problem arises: how can we manage this growth while maintaining, or even improving, system performance? This is called **scalability**.

Let's consider a system with an existing storage solution. As data volumes increase, how can this system be scaled to meet the new demands? There are two main scalability strategies:

- **Vertical scalability** (scale up): this approach involves enhancing the capacity or performance of existing hardware. In practice, this means replacing current resources with newer, more powerful components – such as upgrading to larger memory units or faster processors. Vertical scaling is generally simple to implement, but it has physical and economic limits.
- **Horizontal scalability** (scale out): instead of replacing existing resources, this approach adds multiple units of hardware – even if they are less powerful – by creating a distributed system. Resources are linked together to work as a unified system. The individual components don't necessarily need to be state-of-the-art; the focus is an expansion through quantity, not just quality.

Type	**Pros**	**Cons**
Vertical	- No real changes in how users access the resources - Fast access time to the HW resources	- The larger the storage solution, the more expensive it gets - There's an upper limit to the solution one could get at a given time (a max SSD capacity) - Stop operations → upgrade → resume
Horizontal	- Can scale to very large systems for cheap(er than scale up) - Easy to scale as you go (add or remove resources) if/when needed	- Multiple HW resources connected together → network bandwidth is a very important factor (possibly limiting) - More complex solutions to access all (shared/distributed) HW resources → different from local systems

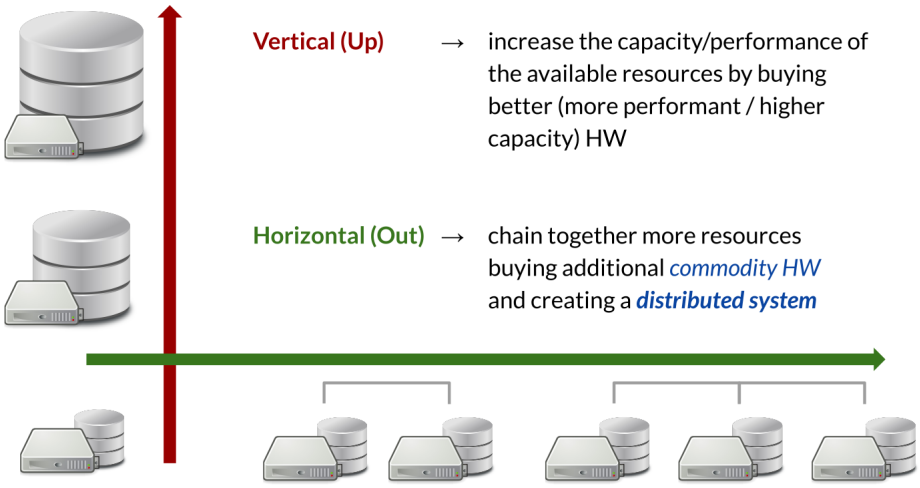

Vertical (Up) → increase the capacity/performance of the available resources by buying better (more performant / higher capacity) HW
Horizontal (Out) → chain together more resources buying additional commodity HW and creating a distributed system

Figure 2.3

Chapter 3

Reliability and Security

3.1 Reliability

So far we've talked about hardware, the physical medium that contains our data. It's important to emphasize that these are *supports*, meaning devices whose main purpose is to store information. As long as the data is safe, it doesn't really matter if the device breaks down beyond repair. What truly matters is the integrity of the data, not the health of the storage unit itself.

Therefore, preserving data stored is important and we must keep it safe from:

1. **Data loss**: if a storage device fails – due to mechanical or electronic malfunctions – all the data it contains is typically lost.

2. **Data corruption**: environmental factors such as voltage drops, high temperatures, or even cosmic radiation can cause a bit flip, which is the unintended inversion of a single bit. Fro HDDs, a bit flip occurs,on average once every 10^{14} bits involved in read/write operations.

   Assuming 1 GB files, we have

   $$\frac{10^{14}\,\text{bits}}{8 \cdot 10^{9}\,\text{bits/file}} = 12500\,\text{files}$$

   that is, on average, 1 file corrupted per 12500 files written.

3. **Transfer errors**: these occur when proper checks are not in place during data copying or transfer procedures.

Reliability is defined as the probability that a system will continue to perform correctly over a given period of time or number of operations.

This concept applies to all hardware, from a bolt to an entire system, and is crucial to understanding and preventing catastrophic issues during operations, such as the loss of data.

An estimate of how reliable an entity is can be achieved by measuring the **mean time to failure** (MTTF) or **mean time between failure** (MTBF) for recoverable systems, shown in Figure 3.1.

The MTTF is the time from the first power-on of a device to its malfunction, while the MTBF is the average time between one failure and the recovery.

A realistic MTTF of a single HDD is $\sim 10\,\mathrm{yr}$. Assuming a flat prior on the probability of a single HDD to incur in a failure in any given day, we expect

$$p = \frac{1}{365 \cdot 10\,\mathrm{yr}} = 0.00027\,\text{failures per day}$$

Considering 5000 independent HDDs, we expect around 1.35 HDD to fail each day. At CERN, with 100000 HDDs, this model predicts 27 HDD failures every day.



Figure 3.1

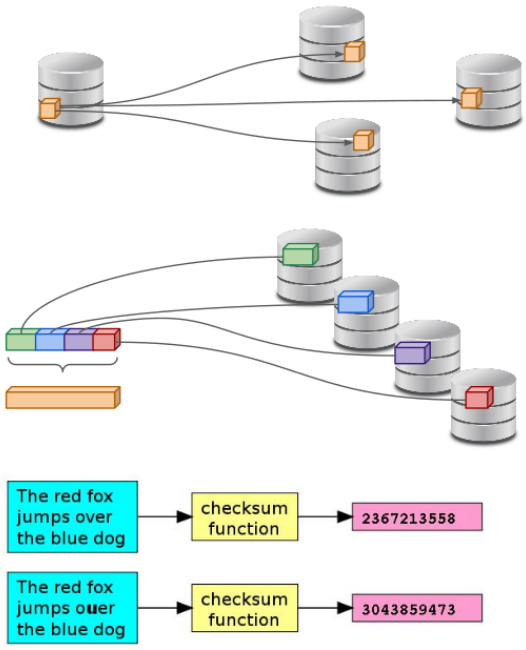


Figure 3.2: From the top: mirroring, striping and checking.

3.1.1 Reliability strategies

Given that, one way or another, data stored in memory devices is constantly at risk, it becomes essential to adopt strategies aimed at ensuring their safe preservation. Data storage reliability (not storage reliability!)[1] can be improved with a number of strategies.

1. **Mirroring**: data is replicated across multiple storage elements (when possible located in different physical palces).

[1]Storage reliability refers to device itself, that is how often it fails? While data storage reliability refers to data storage independently from the device: data are safe through time even if devices fail?

2. **Striping**: data is subdivided in pieces and distributed across multiple storage elements.

3. **Checking**: ensure data consistency by detecting and possibly correcting errors using checksum or parity checks.

The foundation for applying these strategies is the concept of **disk redundancy**, referred to as **Redundant Array of Independent Disks (RAID)**. In this approach, multiple physical storage devices are combined into a single logical unit to enhance reliability and performance.

Note that this is not related to horizontal scalability: the devices are treated as a single unit, made up of different elements working together to increase capacity and performance. This can be implemented in software, but is most commonly deployed via a hardware controller board that daisy-chains multiple disks. Several RAID configurations, known as *RAID levels*, can be implemented.

RAID 0

- It uses disk striping, with no mirroring or checking.
- It uses 2+ disks to store the data, which is striped across all the disks.
- The total capacity is given by the capacity of the smallest disk multiplied by the number of disks.
- The throughput is increased because reading and writing occur simultaneously across all disks.
- Used mostly for performance rather than reliability since losing one disk means losing all data.

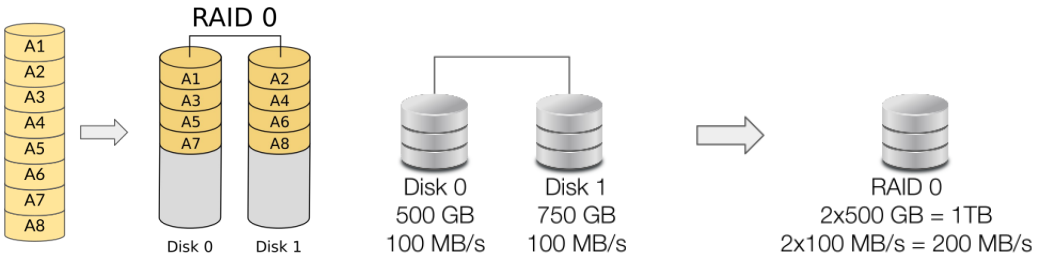


Figure 3.3

RAID 1

- It uses mirroring, no striping.
- It uses 2+ disks to store data.
- Data is mirrored across all disks, meaning it is not subdivided.
- The total capacity is given by the smallest disk. Also the throughput is given by the smallest disk.

- Used for reliability, but has bad performance (slow).
- Data is preserved as long as at least one disk works

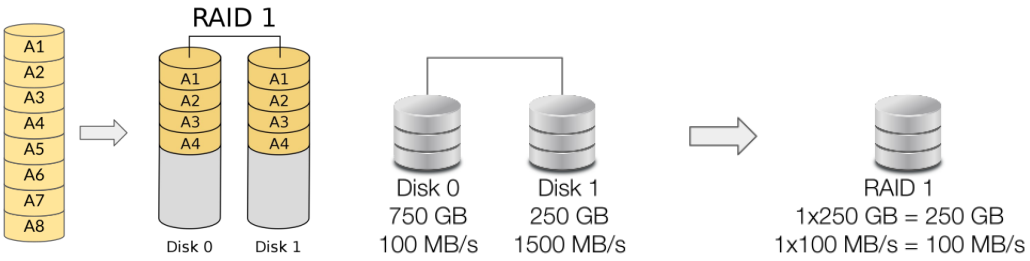


Figure 3.4

RAID 1+0

- It uses mirroring and striping, no checking.
- It uses 4+ disks to store data.
- Data is mirrored (RAID 1) across 2+ disks and striped (RAID 0) across 2+ RAID 1 disks.
- The total capacity is given by the capacity of the smallest RAID 1 unit multiplied by the number of RAID 1 units.
- The throughput in reading is that of RAID 0 (fast), while in writing is that of RAID 0 (slow).
- Good overall performance (close to RAID 0 striping) but improved reliability (from RAID 1 mirroring).
- Data is preserved as long as at least one disk is functional in each mirrored pair

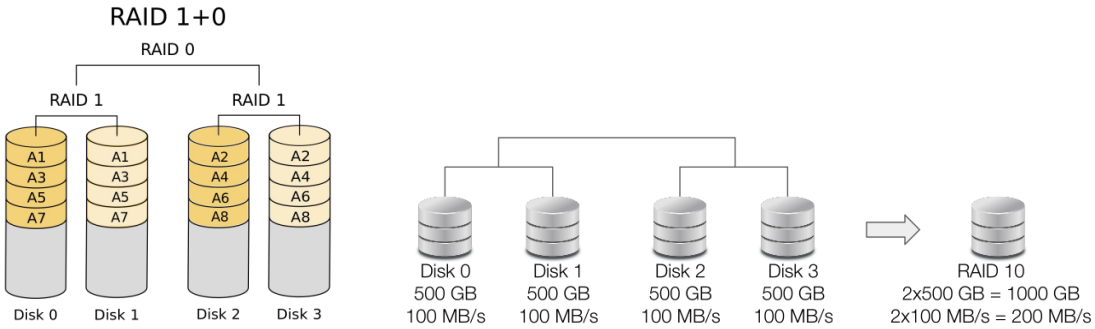


Figure 3.5

A parity information can be stored in dedicated data blocks as an error protection scheme, usually based on `XOR` logic operations. In case of a **single-disk** failure, data can be recovered from a single parity bit information. In case of double-disk failure, it is impossible to recover data. Hence, we can use RAID levels with parity checking.

RAID 5

- It uses striping and parity checking, no mirroring.
- It uses 2+ disks to store data +1 to store parity.
- Data is striped across all the disks.
- The total capacity is given by the capacity of the smallest disk multiplied by the number of disks.
- The throughput is not trivial due to write/read parity information.
- Similar to RAID 0 (for $n-1$ disks), with good overall performance.
- Block parity is distributed across all disks so all disks can participate in satisfying read requests.
- Single disk loss or data error are recoverable.

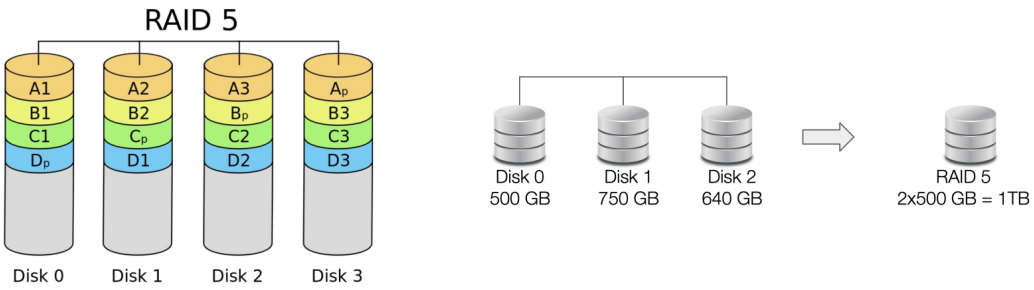


Figure 3.6

RAID 6

- It uses 2+ disks to store data and 2+ to store parity.
- Essentially, it is similar to RAID 5 with two parity information blocks distributed across all disks.
- Even in case of double loss or error, it is possible to recover data.

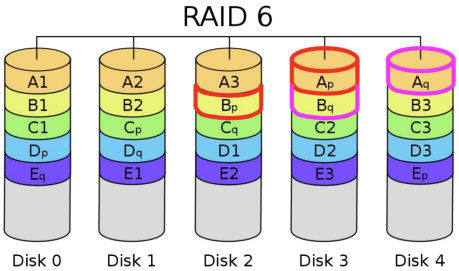


Figure 3.7

3.2 Security

3.2.1 Cryptography

Cryptography is a branch of mathematics that deals with encoding information in such a way that it becomes safe and accessible only to authorized users. It is based on the concept of **security**, which is a key aspect of data management: it verifies user identities, defines what users can or cannot access or perform, and ensures data integrity during transactions, etc.

Cryptography is at the basis of all modern security techniques. The main actions that fall under the theme of security include:

- Protecting data stored on a device.
- Establishing secure connections to a remote server
- Exchanging information between two peers.
- Preventing malicious users from tampering with data.

For the sake of simplicity, we can visualize the main security concepts with the idea of an information exchange between two endpoints, as shown in Figure 3.8.

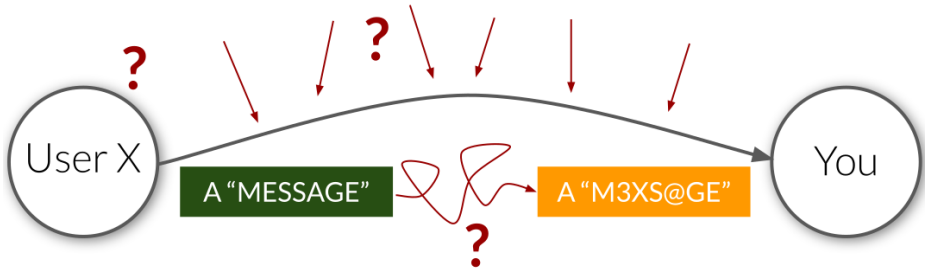


Figure 3.8

In order for this exchange to be considered safe, we must ask ourselves:

- Am I sure who I'm talking to?
- Can anyone else listening to the whole conversation intercept the data? This is one of the most important questions and we will answer it immediately – and answer is yes – it is always possible to intercept information: there is no way to prevent this entirely and we must come to terms with this fact, which forms the basis of the security systems to be implemented.
- Is the message I'm receiving identical to the one that was sent?
- Is the other user responsible for the message they sent?

These questions are fundamental in data management. The main concepts around which cryptography revolves are:

1. **Confidentiality**: ensures that no one can gain knowledge of what you're transferring – even if they listen to the entire conversation.

2. **Integrity**: guarantees that the message has not been altered during transmission.
3. **Authenticity**: allows you to verify that the communication with the entity we believe we're communicating with.
4. **Identity**: allows you to confirm the specific individual behind that entity.
5. **Non-repudiation**: ensures that the individual behind a given asset cannot deny being associated with it.

3.2.2 Authentication and authorization

The last three concepts listed before can be grouped into two closely related aspects:

- **Authentication**: refers to identifying users and validating their identity. This can be achieved in various ways – the most common being customary authentication, that is, username and password authentication. Other methods include certificates, cryptographic keys, SSO, public/private key pairs, and biometric authentication methods.
- **Authorization**: consists of granting a user permission to access or use specific resources.

Usually, authentication occurs when the user connects to a server – known as an **authorization server** – which certifies the login through a specific credential or tool provided by the user.

Only after authentication has taken place, the system proceeds to verify the user's permissions through an *Access Control List* (ACL). This list specifies which resources (stored in the **resource server**) the user can access and which operations the user is allowed to perform.

As already mentioned, once a user has been authenticated and authorized, they become fully responsible for all actions performed under their identity.

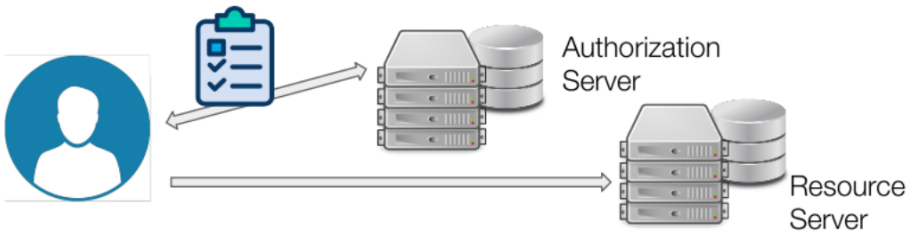


Figure 3.9

3.2.3 Encoding/decoding information

As we already said, all exchanged messages are constantly exposed. Accepting this reality, the only way to prevent information from being extracted is to alter the message in such a way that anyone listening is unable to understand it.

The most basic idea is to apply a sort of mask to the message, known as **key**, which serves to mix the data and then reassemble it. In cryptographic terms, these operations are called **encoding** (ciphering or encryption) and **decoding** (deciphering or decryption).

If the message is represented as a sequence of bits, the key will also be a sequence of bits of the same length. A simple example is the use of `XOR` operation:

- Before the message is sent from A to B, it is encoded using a bitwise `XOR` with the key.
- When B receives the message, he decodes it by applying the same `XOR` operation again with the same key.

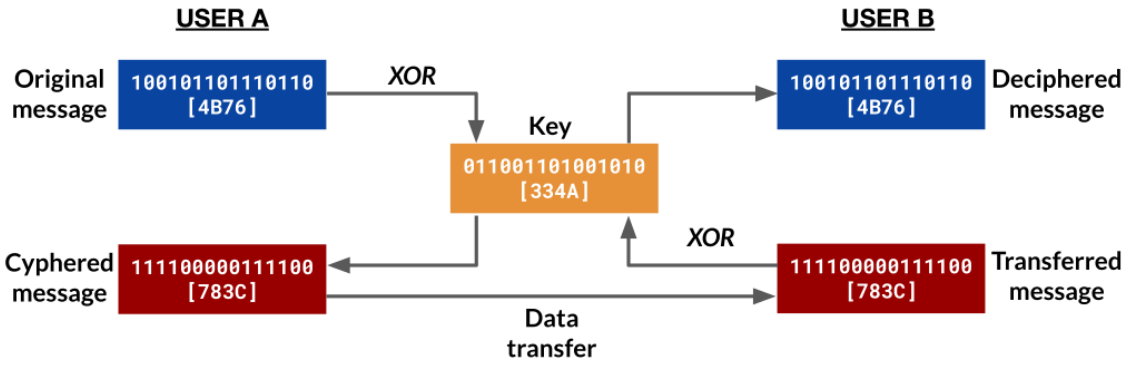


Figure 3.10: Encripting and decripting.

Of course `XOR` is an oversimplification, but the underlying principle is used in practice with much more complex algorithms, such as AES-128, AES-192 and AES-256.

This method is relatively easy to implement, but introduces significant risk: both users must share the same key, which cannot be safely transmitted. The only way to be 100% sure is to share the key in person, but this is clearly inconvenient.

For this reason, this type of cryptography is called **symmetric key encryption** (or private-key encryption) and it is generally not used in high-level security systems, except as an additional layer.

The most widely adopted approach today is **asymmetric key encryption** (or public-key cryptography).

This method uses two distinct keys: one public and one private. Both keys are generated together using algorithms like RSA, ECC, and others, in such a way that is computationally infeasible to derive one key from the other.

Each key can only operate in one direction:

- The public key is solely used to encrypt the data.
- The private key can only decrypt the data encrypted with the corresponding public key.

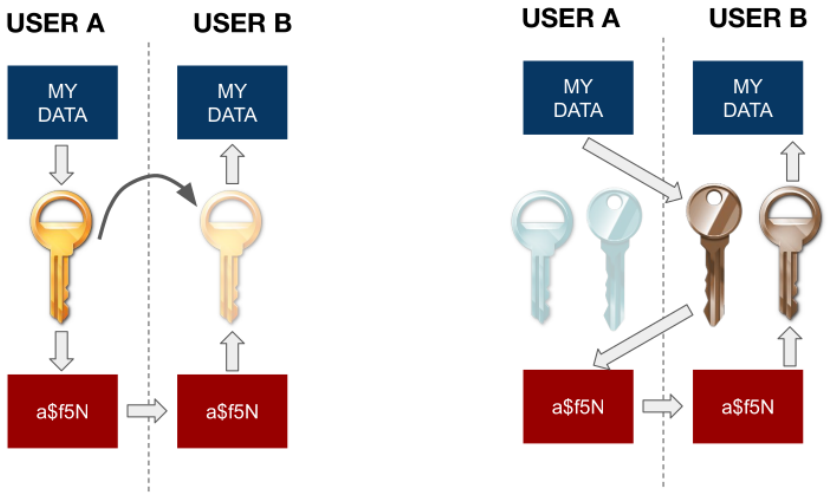
Following this protocol, the user openly shares their public key. Anyone who wants to send them a message will encrypts it using the public key.

The crucial point is that the encrypted message can only be decrypted with the private key, which is exclusively held by the intended recipient.

This encryption/decryption system is significantly slower and more computationally complex, but is also much more secure – to the extent that it is now considered the modern standard.

3.2.4 Hash functions

We have discussed the concepts of authenticity, identity and non-repudiation, which define authentication and authorization, as well as confidentiality, achieved through the encoding



(a) Private-key cryptography. (b) Public-key cryptography.

Figure 3.11

and decoding of messages addressed to their final recipients. Now, we turn to the concept of integrity in the context of exchanged information.

Hash functions are designed for this purpose. By applying the so-called *hashing* to a set of data, the information is rearranged in such way that the original form can no longer be reconstructed.

All hashing-dedicated algorithms – such as MD5, SHA-1*, SHA-2*, produce a result known as a **digest**: a fixed-length output that is the same size for all inputs. Even slight modification of the input will completely change the resulting digest.



Figure 3.12: Digest of similar data.

Hashing algorithms are designed to avoid producing two identical digests for two different raw data. This concept is known as **hash-collision**.

Hashing algorithms are used to:

1. **Mask/protect data**: useful for storing sensitive data, such as usernames and passwords, within computer systems. These types of data cannot be stored explicitly for security reasons, so they are first processed through a hash function, and the resulting digest is what is actually stored in the system.

2. **Data integrity verification**: to ensure that the received data has not been tampered with, it is useful to perform a check using the associated hash – that is, by verifying whether the digest of the copied or downloaded data matches the original one.

3. **Digital signature verification**: due to their uniqueness and irreproducibility, hashes are used to certify archived documents in digital systems through digital signatures. In particular, the document is first hashed, producing a unique digest, which is then encrypted with the sender's private key, creating the digital signature. Anyone with access to the sender's public key can verify the signature by decrypting it and then comparing the resulting hash. If the hash matches, the document is verified and unaltered.

3.2.5 Encryption vs hashing algorithms

Encryption	**Hashing**
Encryption algorithms encode and compress data for confidentiality and to secure transfer.	Hashing algorithms scramble data into fixed-size digest to hide/check data.
They must be fully reversible to allow decryption.	They must not be reversible and must produce a completely different digest for any small difference in data (even 1 bit).
They can be computationally expensive.	They must be computationally efficient and avoid hash collisions.

Table 3.1

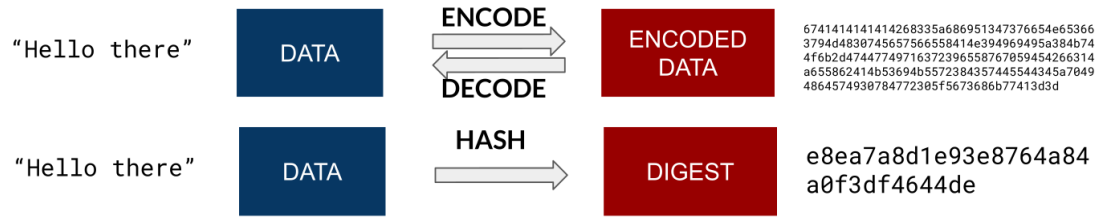


Figure 3.13

Chapter 4

File System

There are three main solutions for storing and managing datasets:

- **Files**: the most well-known and widely used format. Files are blocks of data stored within volumes, managed by a system called a **File System** (FS). When distributed across multiple memory units (horizontal scalability), this becomes a **Distributed File System** (DFS).
- **Databases**: each individual element is accessible at a granular level and can be modified through systems known as **Database Management Systems** (DBMS). When distributed across several memory units, these are referred to as **Distributed Databases**. In such cases, data is typically stored and managed using dedicated systems, most commonly NoSQL solutions.
- **Object Stores**: data is stored in any arbitrary form (file, fixed-length blocks, even full operating systems) inside containers called *buckets*. This solution is used primarily in cloud storage environments.

Let's see file system first.

4.1 Characteristics

The **file system** plays a crucial role in managing and tracking the location of data stored in memory, organizing it in terms of a file hierarchy through directories and subdirectories. Its main characteristics can be summarized in three key aspects:

1. **Data storing**: data is physically stored in blocks of fixed length, scattered across available storage devices (such as HDDs or `NAND` flash cells). The file system is responsible for splitting and allocating files into these blocks, and for keeping track of their exact positions so the system can reconstruct the file when accessed by the user.
2. **Abstraction**: the file system provides an abstraction layer that allows data to be identified and managed as files or directories, using attributes such as file names, paths and metadata.
3. **Low-level operations**: the file system interfaces with the operating system to support low-level operations on storage blocks – such as opening, closing, writing, reading, searching, etc. These operations follow well-defined standards; the most widely

adopted one across modern operating systems is `POSIX` (Portable Operating System Interface for Unix).



Figure 4.1

Depending on the file system, files can be accessed and disk space can be allocated in a several number of ways, including:

- **Sequential access**, **direct/random access** and **indexed access** for file access.
- **Contiguous allocation**, **linked allocation** and **indexed allocation** for space allocation.

Contiguous allocation

In contiguous allocation, files are stored contiguously, meaning that the various blocks composing a single file are physically placed next to each other on the storage device. As a result, file access is sequential, which makes reading the file fast and efficient.

However, the limitations of this allocation method are evident: when attempting to extend or append data to an existing file, it may happen that the space immediately following the file is already occupied by another file. In that case, the file cannot be expanded, since the next block belongs to another file and cannot be overwritten.

For this reason, file systems based on contiguous allocation are best suited for magnetic tapes or archived storage systems, where files are typically written once and rarely modified over time.

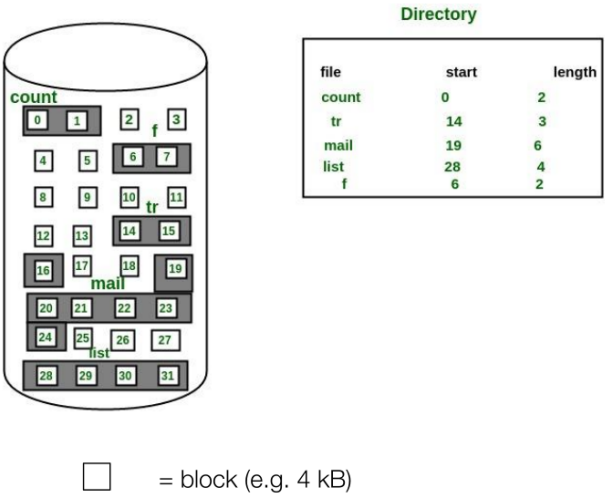


Figure 4.2: Contiguous allocation.

Linked allocation

In linked allocation, the improvement over contiguous allocation is that it is no longer necessary for the blocks of a file to be stored next to each other.

Instead, each block contains a pointer to the location of the next block in the sequence. This allows the file system to jump from one block to another to represent a single file.

As a result, a small portion of each block must be reserved to store the address of the next block. This means that a small amount of memory is sacrificed to allow the file system to navigate and retrieve all parts of the file.

Compared to contiguous allocation, linked allocation optimizes the use of available space, as it does not require a series of contiguous blocks to store a file.

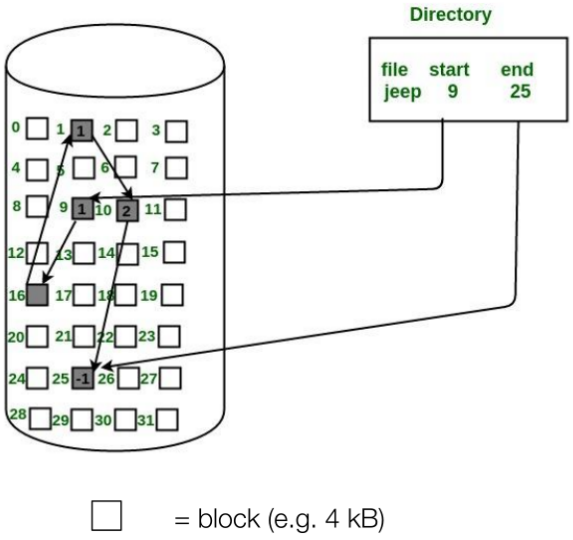


Figure 4.3: Linked allocation.

Indexed allocation

While indexed allocation helps optimize disk usage, it does not optimize operation in time for file-related tasks, such as writing and reading: each time a user requests an operation, the system must start from the first block and sequentially follow pointers through all intermediate blocks until it reaches the target. This makes random access inefficient.

To overcome this limitation, indexed allocation associates each file to a dedicated index block that contains a list of pointers to all the blocks where the file's data is stored.

The structure typically includes:

- File metadata, describing the file itself.
- A series of direct blocks, each containing the addresses of the disk blocks that store the file's data.
- The next few pointers reference indirect blocks. Indirect blocks may be single indirect, double indirect or triple indirect. These blocks contain the addresses of the blocks that in turn store the file's data.

Let's assume we have an indexed file system with 4 kB blocks (Berkeley UNIX FFS); each indirect pointer occupies 4 B. In total the file system has 12 direct blocks and one single block.

How much data can be stored using this file system?

- Each direct data block contains 4 kB, so the total from direct blocks is:

$$12 \times 4\,\text{kB} = 48\,\text{kB}$$

- Each singly-indirect block contains $4\,\text{kB}/4\,\text{B} = 1000$ pointers, and each pointer refers to a 4 kB data block. Thus, each singly-indirect block describes:

$$1000 \cdot 4\,\text{kB} = 4\,\text{MB}$$

- Therefore, the total storage capacity becomes:

$$48\,\text{kB} + 4\,\text{MB} = 4048\,\text{kB}$$

  where about $\sim 1\%$ is due to direct block allocation.

Let's assume there was also a 1 extra doubly-indirect block available. This contains 1000 pointers, each pointing to a singly-indirect block, each containing 1000 pointers. Thus, each doubly-indirect block describes

$$1000 \cdot 1000 \cdot 4\,\text{kB} = 4\,\text{GB}$$

Then, increasing the number of indirect pointer layers, increases exponentially the maximum file size supported by a file system.

Given the block and index sizes in this example, the file system would be capable of storing:

$$n_{\text{direct blocks}} \times 4\,\text{kB} + n_{\text{singly-indirect blocks}} \times 4\,\text{MB} + n_{\text{doubly-indirect block}} \times 4\,\text{GB}$$

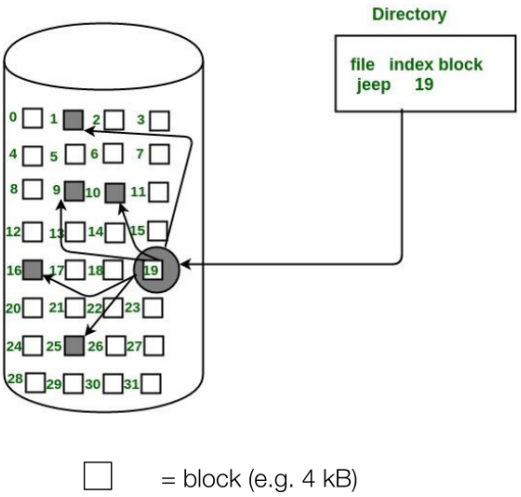


Figure 4.4: Indexed allocation.

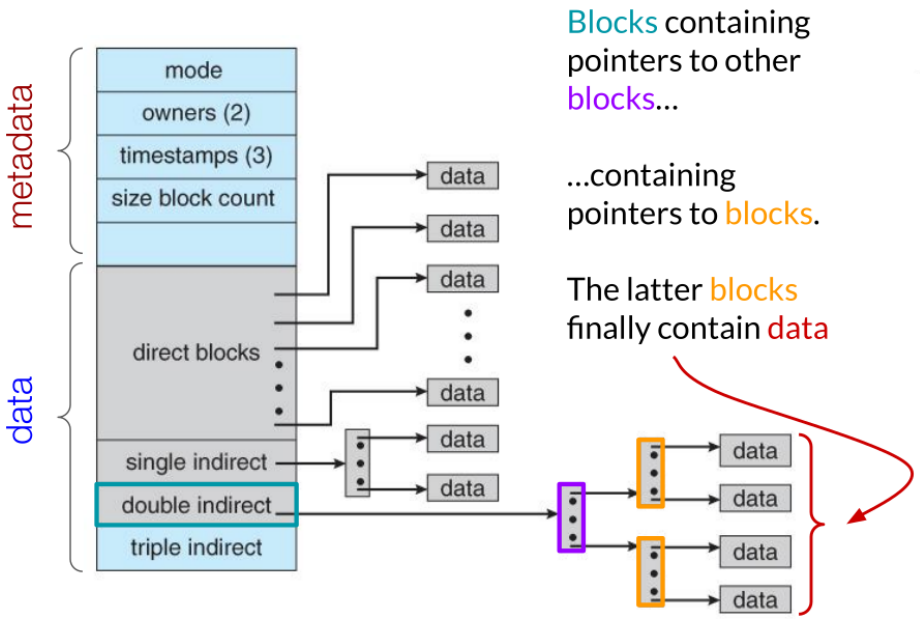


Figure 4.5

4.2 Limits

From the previous reasoning, we understand that a file system is primarily limited by the amount of space it can allocate to a single file. Another important limitation is the total addressable space a file system can manage on a single storage unit.

This constraint arises from the fact that the file system uses a block counter with a limited number of bits to track the location of data blocks – which in turn limits the maximum number of blocks, and therefore the maximum storage capacity that can be handled.

Other differences rely in the metadata that file systems store for each file. The following table compares some of the most widely used file systems.

File System	File	Volume	Filename	Owner	Permissions
FAT32	4 GiB	16 TiB	255 UCS-2 chars	No	No
NTFS	16 EiB	16 EiB	255 chars	Yes	Yes
exFAT	16 EiB	64 ZiB	255 UTF-16 chars	No	No
HFS	2 GiB	2 TiB	31 bytes	No	No
HFS+	8 EiB	8 EiB	255 UTF-16 chars	Yes	Yes
EXT2,3	16 GiB to 2 TiB	32 TiB	255 bytes	Yes	Yes
EXT4	16 GiB to 16 TiB	1 EiB	255 bytes	Yes	Yes

Table 4.1: File system limitations.

Another limitation lies in the fact that a file system cannot provide any information about the content of individual files. As a result, it is up to the user to take advantage of the hierarchical tree structure offered by the system in order to organize and sort the data in the most convenient way. This limitation also means that specific operations, such as applying filters based on file content, cannot be performed directly by the file system.

Pros	Cons
- Can manage huge amounts of data (often EB). - Low-level operations with `POSIX` such as read/write/close are fast. - Offers a hierarchical view of all data. - Single entities (files) can be easily transferred, copied, and shared.	- Complex to define, maintain, and scale a good data hierarchy for large datasets. - The file is the "atomic entity", no internal structure is accessible. - No way to group, select, or filter files based on their content.

Table 4.2: Pros and cons of using file systems for data storage.

4.3 From local to distributed file system

A crucial point we have only partially discussed is how a file system represents acquired data within memory disks – that is, how it maps logical files to physical blocks.

Now, the key question we want to treat concerns the scalability of the system: how well it performs and adapts as the number of bytes in memory increases? This includes evaluating:

- Whether the file system can efficiently manage a growing number of files and larger volumes
- And whether its internal structures (blocks, indexed nodes) can scale without performance degradation.

We already know that in computing, scalability can be either vertical or horizontal. With vertical scalability there are no issues: since the file system interfaces directly with the machine, it can automatically detect and use any newly available memory to allocate files.

However, horizontal scalability may introduce a challenge, as we are now dealing with multiple independent and autonomous machines.

To enable a file system to operate in a horizontally scaled environment, we must move away from traditional **Local File System** (LFS) – the ones previously discussed – and instead adapt **Distributed File Systems** (DFS).

In technical terms, a **cluster** is a collection of computing resources composed of multiple **nodes** – individual machines that work together within the system.

Users typically access the cluster remotely, through a **client program** installed on their personal computer. This client connects to the cluster via a high-speed **communication network**, which links the nodes and serves as the only access point from the outside.

As a result, the user does not interact directly with individual nodes, but instead experiences the cluster as a single, unified system.

DFSs are ideally similar to local file systems, but are extended to environments where both data and processing may be spread out across several nodes (i.e. machines with attached storage).

The DFS itself resides on multiple nodes, but still offers a coherent and uniform view of all data stored across the various disks. Hence, it is shared across the cluster in such a way that the properties and actions we are familiar with are maintained, as long as access to files for reading and writing is preserved.

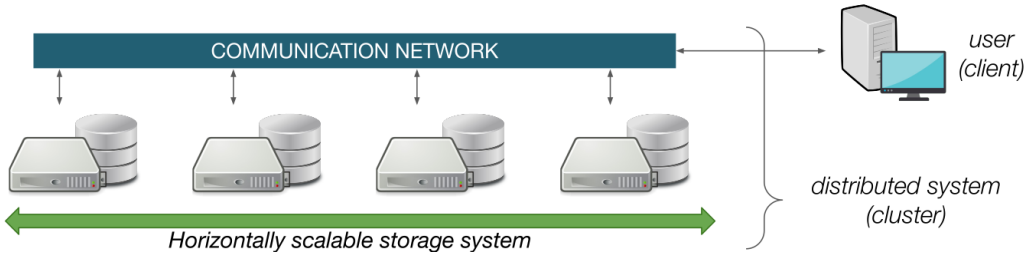


Figure 4.6

4.3.1 Functioning

There are many types of DFSs with different characteristics:

- **NFS** (Network File System): built for `UNIX`-based systems, it is the most widely spread across users and the most appropriate for standard use.
- **AFS** (Andrew File System): appropriate for academic and research purposes.
- **Lustre**: built for HPC (High Performance Computing) in order to maximize the throughput and being able to manage a lot of data.
- **EOS** (Elastic Object Storage): developed at CERN to manage a large amount of unstructured data.
- **GFS** (Google File System): built to manage massive amounts of data across thousands of servers.
- **HDFS** (Hadoop Distributed File System): inspired by GFS, it is optimized for large datasets, high-throughput access, and fault tolerance over commodity hardware.

All DFSs are based on the same fundamental principles as LFSs: data (and metadata) are organized into files, and then split into fixed-size blocks. These blocks are distributed across different physical locations, but remain accessible and traceable through the use of indices and pointers.

However, there are three main differences compared to LFSs:

1. **Block location**: blocks are distributed across different machines. Therefore, to locate a block, it is not enough to specify its memory address – the identity of the machine where the block resides must also be known.
2. **Increased latency**: file operations involve multiple machines, which introduces additional latency. Beyond the delay caused by the mechanical components of individual computers, there is also the latency introduced by the communication network, which is typically higher.
3. **Larger block size**: to mitigate the impact of network latency, block sizes in DFSs are much larger than those in LFSs – typically in the MB range. This allows each read operation to extract a greater amount of data per access, improving efficiency.

   This characteristic is also justified by the fact that DFSs are designed to operate on infrastructures intended to store massive amounts of data. As a result, the block size naturally increases in proportion to the scale of the system, making it significantly larger than in LFSs.

We have discussed how DFS operates in practice to retrieve and allocate data in memory. Now, let's consider the ideal requirements that should guide the design and implementation of such systems.

1. **Transparency**: users should be able to access the system regardless of where they log in from, and perform operations on the DFS as if they were working on a local file system. They should not be exposed to the underlying complexity, the physical location of data, or potential failures of parts of the infrastructure. In other words, the distribution of data across multiple computers should be as invisible as possible – allowing users to work with distributed data as if it were *local*.

2. **Fault tolerance**: the system should continue functioning even in the event of failures – whether temporary or permanent – affecting parts of its infrastructure. It must be resilient to server crashes, network interruptions, and data corruption, preserving both data availability and data integrity.

3. **Scalability**: the system should be able to efficiently scale by adding new servers or storage nodes, even dynamically. It should leverage the increased resources to maintain or improve performance, ensuring that growth in size does not compromise efficiency.

4.3.2 A DFS example: HDFS

HDFS is an open-source DFS developed in 2006 through reverse engineering to emulate GFS, introduced by Google. It was created with the goal of managing and processing large volumes of data using commodity hardware – that is, relatively cheap and widely available machines. Because of this, communication within the system relies on standard TCP/IP protocols.

HDFS is structured around two types of nodes:

1. **NameNodes**: a special node that stores metadata and is responsible for decision-making. It handles client requests for operations such as reading or writing, by either accepting or rejecting them.

2. **DataNodes**: these are the nodes that physically store the data, divided into fixed-size blocks – a typical block size in HDFS is 128 MB. There are many DataNodes in a cluster, and they execute storage and retrieval tasks under the NameNode's direction.

Each client operation involves four steps, shown in Figure 4.7:

1. The client sends a request (e.g. read, write, delete) directly to the NameNode (blue arrow).

2. The NameNode checks permissions and authorizations, then replies with the block locations needed to fulfill the request (purple arrow).

3. The client communicates directly with the DataNodes to perform operations – without further interaction with the NameNode (green arrow).

4. Once the operation is completed, the DataNodes report back to the client, confirming successful completion (orange arrow).

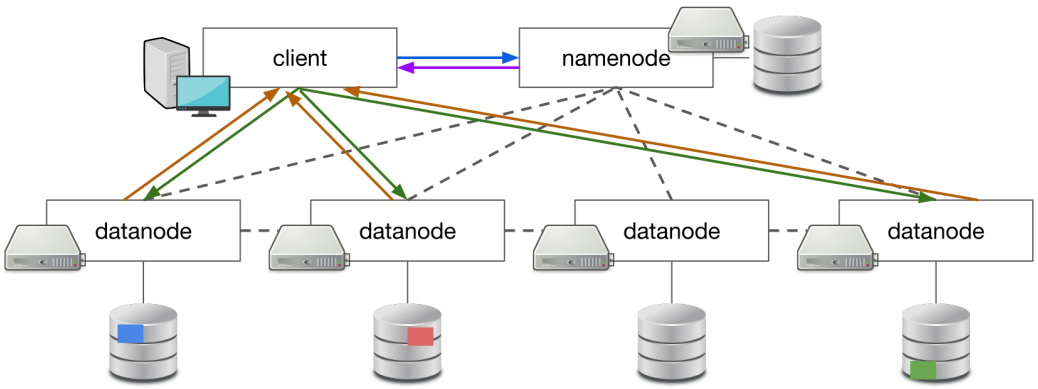


Figure 4.7: HDFS scheme.

Given this system, we can observe that HDFS satisfies the three fundamental ideal requirements previously discussed:

- Transparency is achieved by the fact that the user does not need to deal with the internal steps of file management.

  The user simply starts an operation and waits for a completion response, without needing to worry about how the system handles it internally.

- Scalability is also achieved. HDFS is designed to be highly scalable – adding a new machine to the system and linking it to the NameNode and other DataNodes is a straightforward process, allowing the system to grow dynamically with increasing data needs.

Fault tolerance has to be properly discussed, because we need to break it down into three different types of failures that can affect the system, each requiring specific handling mechanisms:

- **Network instability**: occasionally, network issues may cause one or more DataNodes to become temporarily unreachable, resulting in a *network partition*. To detect these situations, each DataNode periodically sends a *heartbeat* to the NameNode to signal its presence and availability. If the NameNode does not receive the heartbeat within a certain timeframe, the DataNode is considered offline and temporarily excluded from the system.

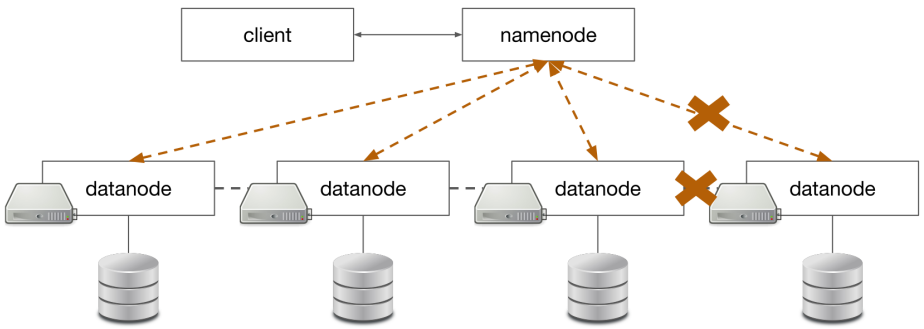


Figure 4.8

- **DataNode failure**: if a DataNode fails permanently, the data stored may be lost. To prevent data loss, each block in HDFS is replicated across multiple DataNodes – typically the default number is three.

  This means that even if one or two copies become unavailable, the system can still access the data from the remaining replica.

  Unlike traditional RAID systems, HDFS achieves redundancy at the cluster level, not through additional physical disks, by distributing copies across different machines.

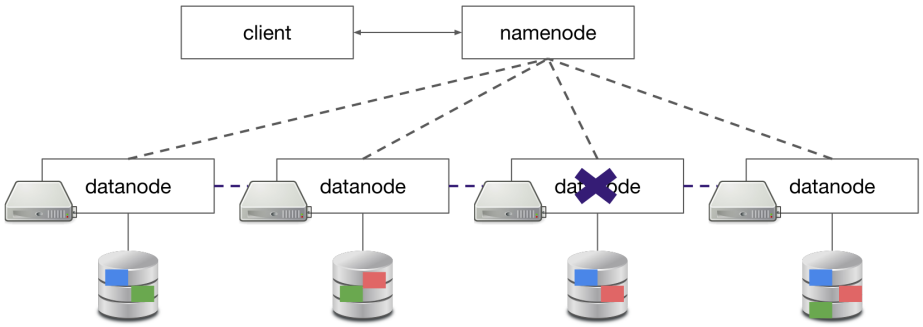


Figure 4.9

- **NameNode failure**: if a NameNode fails, the system loses access to metadata and the interface between clients and the file system. This represents the worst type of failure, known as SPOF (Single Point Of Failure).

  Actually, the data blocks themselves are not lost, but without the metadata, the system no longer knows how to reconstruct files from those blocks.

  To mitigate this risk, HDFS uses a secondary NameNode whose role is to periodically fetch and merge the metadata logs from the primary NameNode, maintaining an up-to-date copy that can be used for recovery purposes (note that it does not provide real-time replication, but periodically checkpoints metadata.).

  However, in the event of a NameNode failure, the system requires manual intervention to replace the failed node with a new one and restore the metadata from backup. This means that the principle of transparency is broken, as the system must be shut down temporarily to carry out the recovery process.

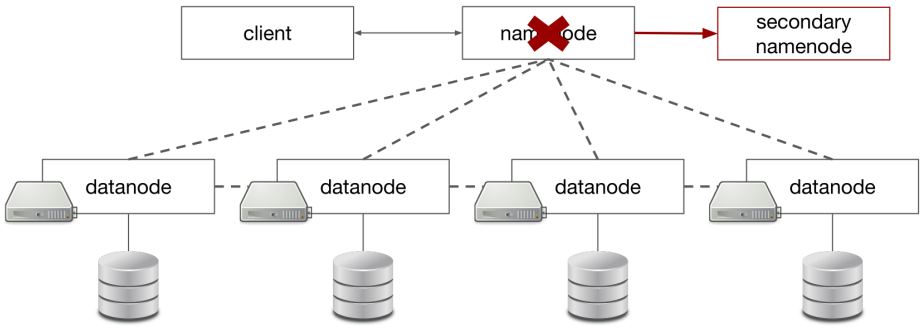


Figure 4.10

4.3.3 CAP Theorem

Having discussed our first example of a distributed system, it is important to highlight the limitations, constraints and tradeoffs that come with such architectures.

A key concept in this context is the **CAP theorem**, also known as Brewer's theorem, formulated by Eric Brewer. This theorem applies to distributed systems – defined as systems composed of multiple interconnected entities that, from the perspective of an external user, behave as a single unified system.

The name CAP stands for the three fundamental properties that are central to the design and operation of distributed systems, both in data storage and processing: consistency, availability and partition tolerance.

1. **Consistency**: every request made by any client to any DataNode must return the exact same copy of the data – that is the most recent and up-to-date version.

   As we know, the same memory blocks are replicated and distributed across different nodes. This means that if two clients simultaneously access the same file, they may interact with different DataNodes – as long as all replicas return the exact same version of the data.

   To guarantee this, the system introduces a synchronization mechanism, often implemented as a waiting queue.

   For example, when a client modifies a data block, the system: registers the modifications, propagates the updates to all replicas and only after all copies have been updated, it allows the client to perform a new operation, such as a new read or write.

   This ensures that all the future accesses – even from different clients and DataNodes – will retrieve the most recent and consistent version of the file.

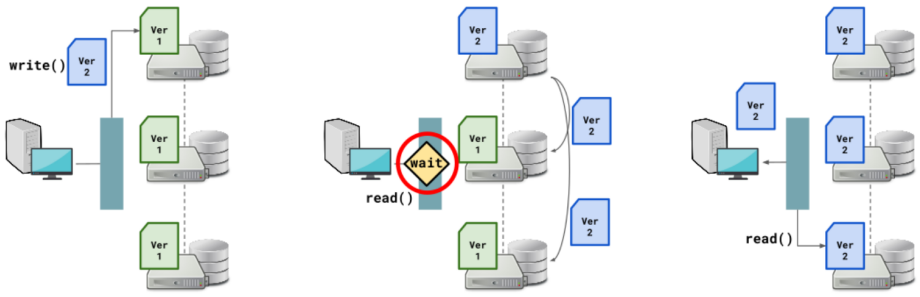


Figure 4.11: Example of consistency during write operation.

2. **Availability**: every request to access, write or update data must receive a response, regardless of the current state of the data or the failure of one or more nodes within the network.

   In other words, every client request must result in an immediate response, even if some nodes are down or unreachable.

   To comply with this principle, the system must be capable of:

   - Handling simultaneous requests from multiple clients, minimizing latency and avoiding any delay in the response.

- Ensuring a high level of fault resilience by maintaining a sufficient distribution of data across multiple nodes, so that clients can still access the same file even if some DataNodes become unavailable

For the first requirement, data must be replicated and distributed effectively across the cluster to allow concurrent access to the same resources. For the second, communication between clients and the network must be optimized to eliminate waiting times and ensure responsiveness at all times.

To emphasize this last aspect, let's consider the following example: a client writes to a file, modifying it to a new version and then immediately attempts to read the same file. According to the principle of availability, every node containing a replica of that file must respond as quickly as possible.

However, due to the timing of the operation, the response may come from:

- A node that already contains the updated version
- or a node that still holds the previous version.

In this case, it becomes impossible to guarantee which version the client will read – because the outcome depends on a race condition between the write propagation and the read request.



Figure 4.12: Example of availability during write operation and read request.

3. **Partition tolerance**: the system must continue to operate despite failures in the communication between nodes, known as *network partitions*.

   As we saw in the previous example, when we attempt to write a file updating it to a new version, it is essential that at least one DataNode containing the relevant block is available to the client.

   As we will see shortly, if this condition is met, we are faced with a trade off between two possible strategies:

   - We may choose to wait for a consistent read, ensuring that all clients receive the same updated version of the file across the entire system – prioritizing consistency.
   - Or we may prefer to return the fastest possible response, accepting the possibility of a race condition, and thus sacrificing consistency for availability.

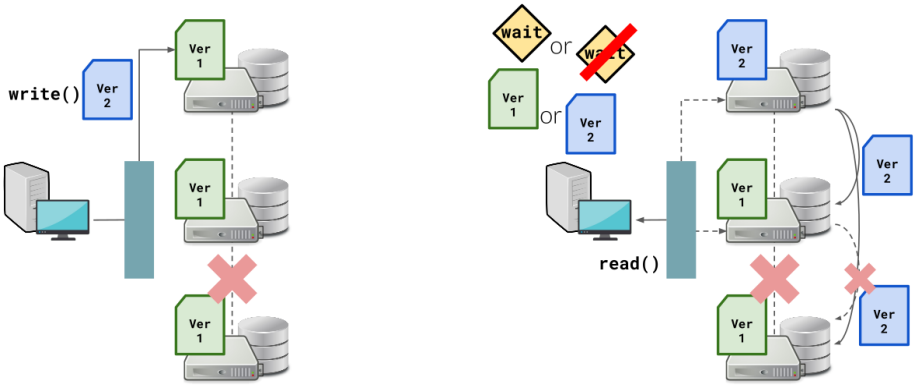


Figure 4.13: Example partition tolerance during writing operation.

The **CAP theorem** states that it is impossible for a distributed system to simultaneously provide more than two out of the three CAP guarantees (consistency, availability and partition tolerance). In distributed systems, network partitions are unavoidable. Therefore,

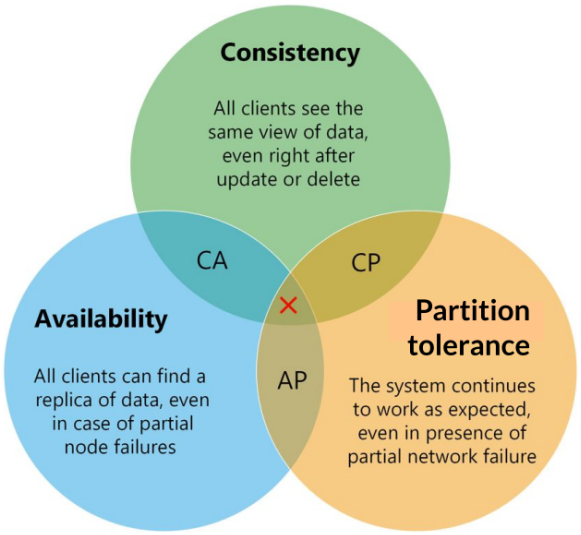


Figure 4.14

to maintain system functionality during a partition, we must choose between prioritizing consistency or availability:

- **AP systems** prioritize availability: they always return a response to client requests, even during network partitions, but they may be outdated.
- **CP systems** prioritize consistency: they guarantee that every read returns the most recent value, but it may be delayed. In other words, clients may have to wait if consistency cannot be ensured.

The choice between these two approaches depends on which property the system is designed to prioritize.

In high-performance computing systems, such as Lustre, the priority is often fast access to data that is constantly being updated. In such cases, the systems adopt an AP model

(availability + partition tolerance) allowing quick responses even if temporary inconsistencies in data version occur. On the other hand, in systems that require a higher level of overall data reliability, such as banking or financial systems, it is preferable to adopt a CP model, which ensures that the system always reflects the most up-to-date state of the data, with no discrepancies in read results across different devices.

As already discussed, the AC combination is not feasible in distributed systems, as it does not account for partition tolerance. However, this model can still be adopted in vertically scalable systems, where there is no need to add new nodes to distribute data and computation. Instead, the system is expanded by integrating increasingly powerful memory and processing units on a single machine. This allows maintaining both availability and consistency, as partitioning does not occur in such architectures.

Chapter 5

Databases

A **database** can loosely be defined as a set of records (often related) organized and managed by dedicated management systems.

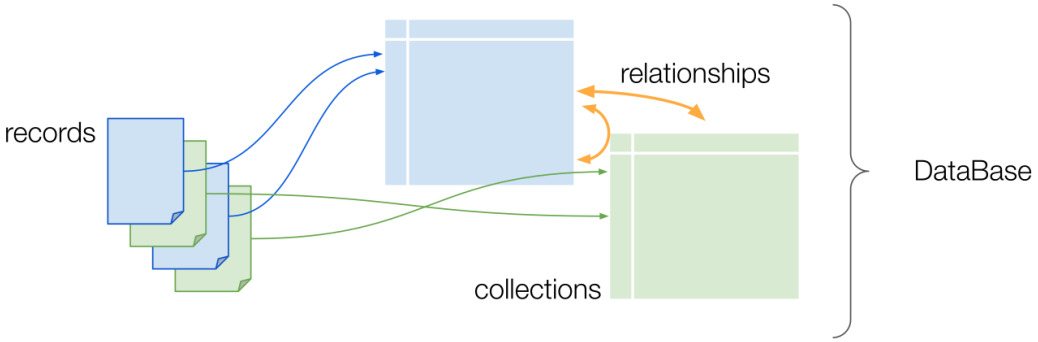


Figure 5.1

Usually when we use the term "database", we are implicitly referring to three related concepts:

- **Records**: the basic unit of information, typically representing a single entity or data point.
- **Collection**: logical groupings of records that share the same structure or context.
- **Relationships**: the connections that may exist both among records within the same collection and between records from different collections.

The **Database Management System** (DBMS) is a software system that provides a number of basic functions to operate on the database, including

- Defining / creating data.
- Querying / selecting data.
- Manipulating / processing data.

It is important to highlight the main differences between data management through a file system and through a DBMS:

1. **Granularity of access**: the scale at which the two systems operate is fundamentally different.

   With a file system, the user can only access entire files, meaning the entire content of the file must be retrieved.

   In contrast, a DBMS allows operations to be performed at the level of individual records, enabling much more granular and efficient data access.

   For example, accessing single events within a dataset, instead of loading all records in a file – leading to significant optimization over file-based operations.

2. **Data organization and structure**: in file systems, data is often scattered across multiple files and directories, with no inherent structure or relationships between pieces of data.

   In a DBMS, all data is logically organized within a single entity – the database – which also includes the metadata and the relationships between records.

   All data (and metadata) of a dataset is managed by the DBMS – e.g. data and detector configuration, banking wire transfers and users' data. All dataset information is accessible, including relationships between records.

3. **Efficient data reuse and filtering**: in data analysis workflows using file systems different algorithms applied to the same dataset often require separate copies of the data – leading to redundancy and wasted memory.

   With a DBMS, it is possible to filter or select subsets of the data without the need to replicate it.

   Little-to-no replication is required to handle multiple *views* of the same data. Thus, the whole dataset can be stored once, and different subsets can be accessed through queries (filters) based on relationships across records, without occupying additional memory.

Therefore, databases tend to improve the management of large datasets over (distributed) file systems.

5.1 Databases models

In databases, it is common to distinguish between the description of the data and its actual content. This distinction is made by introducing two key concepts:

- **Database model** (or database schema): represents the structure of the database – that is, the description of the data, its organization and the relationships between the various elements of the dataset.

- **Database instance** (or database state): it represents the actual data contained in the database at a specific moment in time. It represents a snapshot of the database's content based on the current values stored in the tables defined by the schema.

The DBMS is responsible for defining the schema, as well as updating and manipulating the database instance. It is often simplified as referring to "database" to indicate both the database and the DBMS.

At this stage, we focus primarily on the model – in other words, the structure of the database.

It is clear that structured data naturally fits into an organized schema. For this reason, different types of databases are offered depending on whether the data is structured, semi-structured, or unstructured.

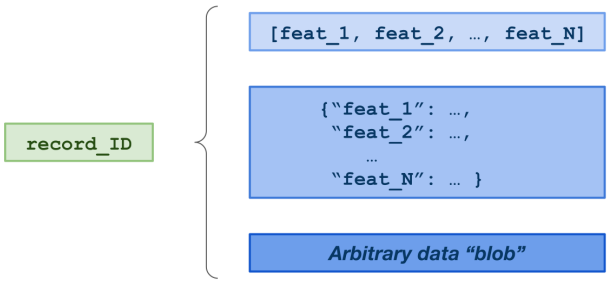


Figure 5.2: Type of records.

As previously mentioned, the schema – directly provided by the DBMS – is considered one of the key factors that differentiate database types.

Other distinguishing aspects include:

- The type of access: single-user or multi-user.
- The architecture: centralized in a single database or distributed across multiple databases.

The most relevant categorization from the user's perspective is based on the data model, i.e. the way each record is represented.

One of the most common mechanisms used by data models to access individual records is through the use of **keys**. Each record can be thought as a **value** indexed by a unique identifier, the key.

This method is used in:

- **Relational Databases**, a widely spread standard for structured data; also known as **SQL databases** because they rely on the SQL language to manage and query data.
- **Key-value databases**, suitable for semi-structured and unstructured data, where each record is stored as a unique key-value pair.

In general, we distinguish between two major classes of data models:

- **Relational databases** (SQL): used for structured data.
- **Non-relational databases** (NoSQL): designed to handle semi-structured and unstructured data, and referred also as **NoSQL**, meaning **Not only SQL** – indicating that the data model does not necessarily follow a rigid, predefined structure.

5.2 Relational databases

The first software systems for managing relational databases were developed in the 1970s, following the introduction of relational algebra by Ted Codd.

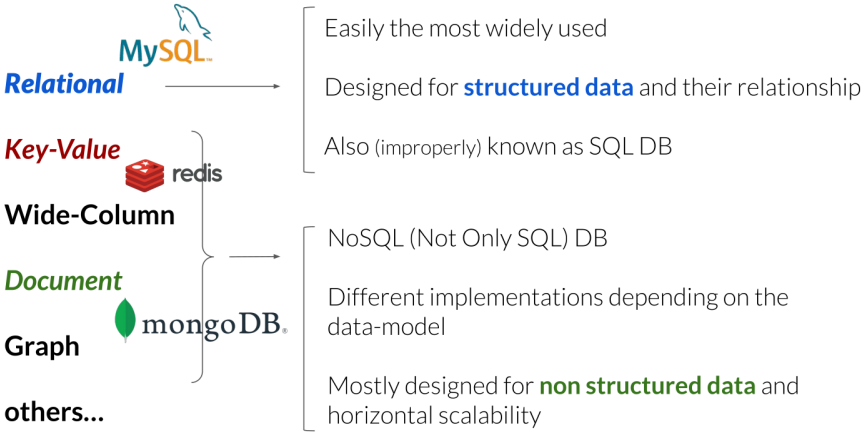


Figure 5.3

MySQL is one of the most widely used today because it is an open-source management system.

As we have discussed, RDBs require a well-known and predefined schema. For this reason, they are well-suited for structured data, but not ideal for other types of data.

An RDBMS organizes data into **tables**, where

- Columns define the attributes (or categories) of the data.
- Rows represent the individual records (or tuples).

This tabular structure enables the mapping of relationships between records, even across different tables.

Let's consider an example of a database containing the information about bikes, with three simple records.

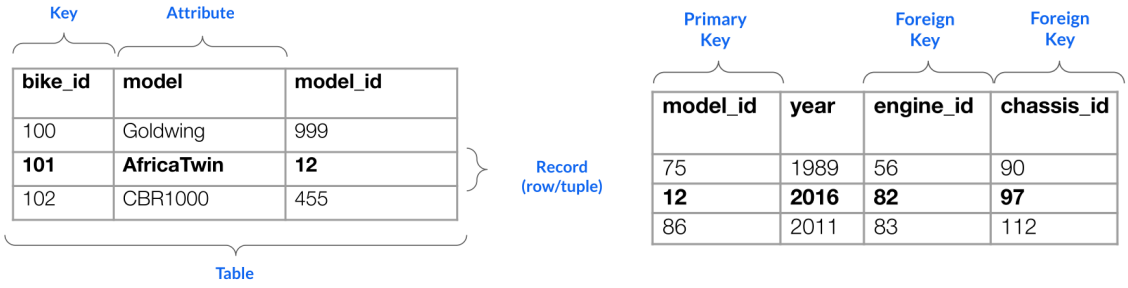


Figure 5.4

We can clearly distinguish three key components:

1. **Record**: each row of the table is a distinct record.
2. **Attribute**: columns of the table representing specific properties.
3. **Primary key**: which is a specific attribute (in general can be a combination of attributes) used to uniquely identify each record. It must satisfy two conditions:
   - **Uniqueness**: that is, no two tuples can have the same primary key value.

- **Minimality**: the key must be composed of the minimum number of attributes necessary to maintain uniqueness.

Individual tables allow records to be ordered and identified by listing all their information based on the primary key. When assessing relationships across multiple tables, we do refer to the primary key and the **foreign key(s)**, which is an attribute or a combination of attributes acting as the link to the key to other tables; in other words, it refers to the primary key of another table.

This mechanism allows data to remain logically linked without needing to duplicate information across tables.

From the previous example, we can expand the original *bikes* table by introducing additional related tables linked through foreign keys.

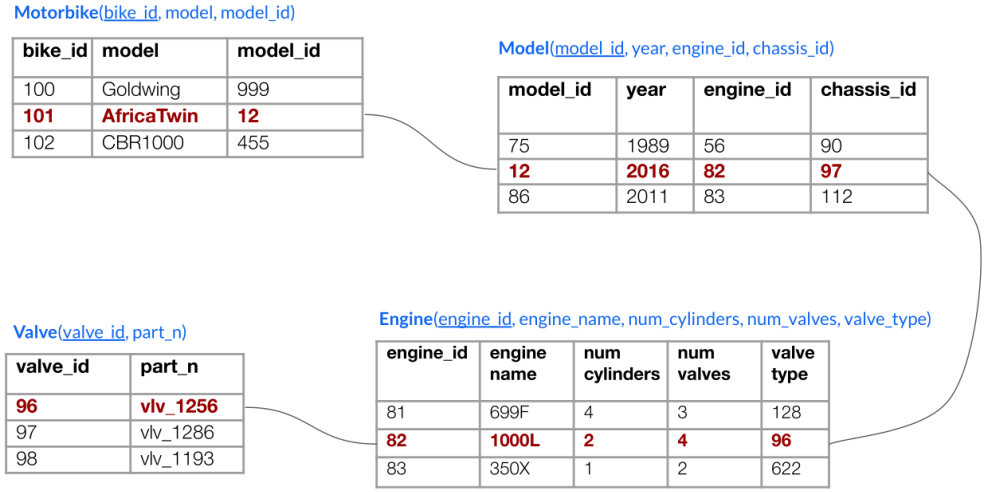


Figure 5.5

In this case, we have one primary (obvious) and more foreign keys.

Each set of operations on RDBs is referred to as a **transaction** and each of them must satisfy four criteria summarized by the acronym **ACID**:

1. **Atomicity**, also referred to as **all-or-nothing**, meaning that either all operations in a transaction are executed completely or the entire transaction is aborted and no changes are applied to the database.

2. **Consistency** (not to be confused with the one from CAP): ensures that each transaction moves the database from one valid state to another. A valid state is one in which all integrity constraints defined on the database are satisfied.

   For example, if an attribute is constrained to contain a fixed-length string of n characters, this rule must hold before and after every transaction involving the table that contains the attribute.

3. **Isolation**: ensures that every transaction, even when executed concurrently with multiple other users, must generate the same results as if it were executed in a single-user environment, unaware of other simultaneous operations.

   This property guarantees that concurrent transactions do not interfere with each other, preserving the global consistency of the database and allowing each user to observe coherent and predictable results.

4. **Durability**: after a transaction, the database state persists until is changed by another transaction.

5.2.1 SQL

The standard language used by all DBMSs managing relational databases is SQL, originally called SEQUEL (Structured English Query Language).

It is an older language, developed in 1974 and officially standardized in 1986, when it was adopted by ISO.

SQL is a **declarative language**, meaning that it mainly describes *what* should be done, rather than *how* it should be carried out. It can be divided into three main parts:

1. **Data Definition Language** (DDL) → to define the DB schemas.
2. **Data Manipulation Language** (DML) → to query and modify data.
3. **Data Control Language** (DCL) → to perform authorization and access control.

In the following, the main statements used to work with databases are presented.

Creation of databases and tables

Every instruction in SQL must end with a semicolon. To display the databases currently stored in memory, the following statement can be used:

```
SHOW DATABASES;
```

This command will return a list of available databases.

In order to create a database, the `CREATE` statement can be used, followed by the name of the database. To avoid potential issues, it is always possible to specify that the database should be created only if it does not already exist:

```
CREATE DATABASE IF NOT EXISTS MyDataBase;
```

To work with a specific database, the`USE` statement is used, followed by the name of the database:

```
USE MyDataBase;
```

While working with a database, it is possible to display the tables it contains by using:

```
SHOW TABLES;
```

In order to create a table, a schema is required. It is necessary to specify the names of the attributes together with their associated data types. Upon creation of a table, the instance (the actual data contained) is empty. SQL provides different types of variables, some of these shown in Table 5.1.

The following is an example of how to create a table:

```
CREATE TABLE Users (
    UserID varchar(30),
    BadgeNum int(16),
    FirstName varchar(255),
    LastName varchar(255),
    Age int,
    OtherAttr float
);
```

Table 5.1: Examples of SQL variable types

`Data Type`	**Description**	**Limit**	**Default**
`char(n)`	Fixed-length string of length n	$n = 2^8$	$n = 1$
`varchar(n)`	Variable-length string with maximum length n	$n = 2^{16}$	–
`boolean`	Boolean value	$\{0, 1\}$	–
`smallint`	Integer number	$[-2^{15}, 2^{15} - 1]$	–
`int`	Integer number	$[-2^{31}, 2^{31} - 1]$	–
`float(n,d)`	Floating-point number with n total digits and d decimal digits	$(n, d) = (24, 23)$	$(n, d) = (10, 2)$
`double(n,d)`	Floating-point number with larger precision	$(n, d) = (53, 52)$	$(n, d) = (16, 4)$
`date`	Date in format YYYY-MM-DD	–	–
`time`	Time in format hh-mm-ss	–	–
`datetime`	Date and time in format YYYY-MM-DD hh-mm-ss	–	–

Keys

To specify the primary key, the `PRIMARY KEY` option must be written next to the desired attribute:

```
CREATE TABLE Users (
    UserID varchar(30) PRIMARY KEY,
    BadgeNum int(16),
    FirstName varchar(255),
    LastName varchar(255),
    Age int,
    OtherAttr float
);
```

If the key is composed of multiple attributes, it is not necessary to write `PRIMARY KEY` next to each attribute. Otherwise, SQL would interpret each attribute as an independent primary key, resulting in the following error:

```
CREATE TABLE Users (
    UserID varchar(30) PRIMARY KEY,
    BadgeNum int(16) PRIMARY KEY,
    FirstName varchar(255),
    LastName varchar(255),
    Age int(16),
    OtherAttr float
);

ERROR 1068 (42000):  Multiple primary key defined
```

To define a primary key composed of multiple attributes, it is necessary to add an additional line specifying the attributes inside parentheses:

```
CREATE TABLE Test (
    AttrOne varchar(255),
    AttrTwo varchar(255)
    PRIMARY KEY (AttrOne, AttrTwo)
);
```

To indicate a foreign key, it is necessary to specify the attribute inside parentheses and use the `REFERENCES` statement, followed by the name of the referenced table and, in parentheses, the name of the corresponding primary key.

```
    CREATE TABLE Table1 (
    ThisKey varchar(8) PRIMARY KEY,
    Feat1 varchar(30)
);

    CREATE TABLE Table2 (
    PrimKey varchar(10) PRIMARY KEY,
    Feat2 int(16),
    ForKey varchar(8) UNIQUE,
    FOREIGN KEY (ForKey) REFERENCES Table1(ThisKey)
);
```

Constraints

Each attribute can be associated with one or more constraints, indicated by keywords in uppercase placed after the variable type associated with the attribute. The constraints that can be assigned to attributes are:

- `UNIQUE`, which ensures that each record associated with the attribute has a unique value. For example, when defining a foreign key, it is typical that the referenced attribute is also unique.
- `NOT NULL`, which specifies that an attribute cannot assume `NULL` values. A `NULL` value appears when a record does not specify a value for that attribute. To enforce this constraint, simply add the `NOT NULL` keyword.
- `DEFAULT`, which allows you to set a default value when the user does not specify one during record insertion. The default value must, of course, be compatible with the data type defined for the attribute. The syntax requires using the `DEFAULT` keyword followed by the desired value.
- `CHECK`, which allows setting a condition that the values of the attribute must satisfy. If the condition is not met, an error will be returned. The condition must always be enclosed in parentheses and preceded by the `CHECK` keyword.

Here is an example of applying constraints to the `Users` table:

```
CREATE TABLE Users (
    UserID varchar(30) PRIMARY KEY,
    BadgeNum int(16) NOT NULL UNIQUE,
    FirstName varchar(255) NOT NULL DEFAULT '',
    LastName varchar(255)NOT NULL DEFAULT '',
    Age int(16) DEFAULT 18,
    OtherAttr float CHECK (OtherAttr>0)
);
```

To inspect a table and view its attributes along with their associated properties, it is possible to use the `DESCRIBE` command followed by the table name.

```
DESCRIBE Users;
```

Field	Type	Null	Key	Default	Extra
UserID	varchar(30)	NO	PRI	NULL	
BadgeNum	int	NO	UNI	NULL	
FirstName	varchar(255)	NO			
LastName	varchar(255)	NO			
Age	int	YES		18	
OtherAttr	float	YES		NULL	

Alteration of Tables

Once the schema of the table has been created, it can be modified, keeping in mind the various restrictions imposed by constraints. It is also important to remember that altering the structure of a table affects all the records contained in it. This operation can take a significant amount of time.

In the case where a new NOT NULL attribute is added, all records contained in the table must now contain a valid value; therefore, a DEFAULT must be specified.

In fact, one of the main challenges when working with relational databases is maintaining their consistency. For this reason, a specific term has been introduced for the creation and maintenance of RDBs: **database normalization**.

Each alteration of a table is performed by starting with the `ALTER TABLE` statement, followed by the name of the table. To add a column, the `ADD COLUMN` statement is used, followed by the name of the column, the associated variable type, and any constraints on the attribute:

```
ALTER TABLE Users
ADD COLUMN Role char(1) DEFAULT 'A' CHECK
    (Role IN ('A', 'B', 'C'));
```

To set a default value for an existing attribute, or reset it in case it already has one, we use a syntax that involves the new statements `ALTER` and `SET DEFAULT`:

```
ALTER TABLE Users
ALTER Role SET DEFAULT 'B';
```

To add a constraint to an already existing attribute, it is possible to use the statement `MODIFY`. In SQL, modifying an attribute implies redefining it; therefore, it is necessary to re-write — beside the name of the column — the type of variable:

```
ALTER TABLE Users
MODIFY Role char(1) NOT NULL;
```

If a table has been created without specifying the primary key, it is possible to do so with the following:

```
CREATE TABLE AddressBook (
    TelephoneNumber char(10),
    User varchar(30)
);

ALTER TABLE AddressBook
ADD PRIMARY KEY (TelephoneNumber, User);
```

To define a foreign key, instead, the syntax is more complex:

```
ALTER TABLE AddressBook
ADD CONSTRAINT FK_user
FOREIGN KEY (User) REFERENCES Users(UserID);
```

`FK_user` is the name of the constraint given directly by the user, which can also be removed with the following syntax:

```
ALTER TABLE AddressBook
DROP CONSTRAINT FK_user;
```

In case the name is forgotten, it is possible to view all information about the table by using `SHOW CREATE TABLE`:

```
SHOW CREATE TABLE AddressBook;
```

Now, let's see how to drop an attribute:

```
ALTER TABLE Users
DROP COLUMN Role;
```

To delete an entire table, it suffices to use:

```
DROP TABLE IF EXISTS Users;
```

Deleting a table implies not only the removal of all records it contains, but also the possible deletion of links to other tables through foreign keys, potentially rendering the entire database unusable. Therefore, caution is advised when using this command.

SQL offers all CRUD operations:

- **C**reate → `INSERT` → insert new records in a table.
- **R**ead → `SELECT` → performs queries on the database.
- **U**pdate → `UPDATE` → modifies records.
- **D**elete → `DELETE` → delete records.

In addition to the list of basic operations, it offers a number of additional features, e.g.:

- Conditional queries.
- Table joins.
- Aggregations.
- Renaming and aliases.
- Range operations.
- Views.

Let's see in detail these operations.

Data insertion

For each record insertion, it is necessary to specify the attributes associated with the tuple. Attributes that can be `NULL` or have a `DEFAULT` can be omitted:

```
INSERT INTO Users (UserID, BadgeNum, FirstName, LastName, OtherAttr)
VALUES ('usr:00001', 100, 'Rei', 'Ayanami', 1.8);
```

Multiple records can be inserted separated by a comma:

```
INSER INTO Users (UserID, BadgeNum, FirstName, LastName, OtherAttr)
VALUES
('usr:00002', 101, 'Asuka', 'Soryu␣Langley', 3.5),
('usr:00003', 102, 'Shinji', 'Ikari', 2.9),
('usr:00004', 103, 'Toji', 'Suzuhara', 4.7),
('usr:00005', 104, 'Kaworu', 'Nagisa', 3.0);
```

Record Update

The statement that allows modifying records individually or collectively is `UPDATE`. It is followed by the `SET` statement and then by a condition that specifies where the modification must be applied, using the `WHERE` statement:

```
UPDATE Users
SET Role = 'C'
WHERE (LastName = 'Ayanami');
```

It is now useful to define the concept of a **wildcard**, which refers to two special characters used to extend the conditions applied by the `WHERE` statement when working with strings:

- _ is a *char wildcard*.
- % is a *string wildcard*.

They represent any possible value that can be matched either as a single character or as a sequence of characters, and must be used in combination with the `LIKE` operator. For example, by writing:

```
UPDATE Users
SET Role = 'B'
WHERE (FirstName LIKE '%o%');
```

for all records whose `FirstName` attribute contains a string with the character 'o', the `Role` attribute will be set to 'B'. If instead we write:

```
UPDATE Users
SET OtherAttr = 5.0
WHERE (FirstName LIKE '__i%');
```

the `OtherAttr` attribute will be set to 5.0 for all records whose `FirstName` attribute corresponds to a string whose third character is 'i'.

Record deletion

Record deletion proceeds in the same way as modifications and allows either the removal of all records from a table or the removal of a selection of records based on a condition. This is done through the `DELETE FROM` statement followed by the name of the table. For example, the deletion of records whose second-to-last character of the `LastName` is 'p' is done as follows:

```
DELETE FROM Users
WHERE (LastName LIKE '%p_'
```

Data selection

The `SELECT` statement is the most powerful and widely used command in the SQL language, as it allows the extraction of specific records according to conditions specified by the user — this is known as a **query**. `SELECT` enables querying an entire table and extracting records that satisfy specific conditional statements.

The first practical use of this function is visualization; for example, to display the entire table, we can write:

```
SELECT * FROM Users;
```

where $*$ is the notation used to indicate *all* the attributes of a table without applying any constraints.

It is also possible to impose a limit on the number of visualized records by using `LIMIT`:

```
SELECT * FROM Users
LIMIT 3;
```

To display only specific attributes, it is possible to specify them by replacing the asterisk:

```
SELECT FirstName, BadgeNum FROM Users;
```

Moreover, it is possible to display all distinct attributes without repetitions (for example, in the case of homonyms); to do this, one must use the `DISTINCT` keyword together with the `SELECT` statement:

```
SELECT DISTINCT FirstName FROM Users;
```

Alternatively, `DISTINCT` can sometimes be used to obtain similar results as `GROUP BY` when simple deduplication is sufficient.

Another possibility is to sort the table based on an attribute; if not specified, the sorting will be ascending by default, but it is also possible to sort in descending order by using the `DESC` option:

```
SELECT FirstName FROM Users
ORDER BY FirstName;
```

It is possible to specify additional conditions through the `WHERE` clause:

```
SELECT FirstName, BadgeNum FROM Users
WHERE OtherAttr > 0;
```

Another possibility for visualization consists in defining a special table, called a `VIEW`, by selecting specific attributes and imposing conditions that the records must satisfy. Let's see an example:

```
CREATE VIEW OneThree AS
SELECT UserID, BadgeNum, LastName
FROM Users
WHERE (OtherAttr BETWEEN 1 AND 3)
    OR (Role IN ('A', 'C'));
```

in which `BETWEEN` checks if in range and `IN` checks in list of values. `OR` is a boolean operator.

`OneThree` will appear in the list of tables when using the `SHOW TABLES` statement, and its content can be displayed by writing:

```
SELECT * FROM OneThree;
```

To display the characteristics of a view, one may write:

```
SHOW CREATE VIEW OneThree;
```

Each modification made to the main table `Users` will be reflected in the visualization of `OneThree`. At the same time, it is possible to treat `OneThree` as a table in all respects — inserting, modifying, and deleting records — with all changes being reflected back on `Users`.

It is also possible to insert a record into the view that does not satisfy the visualization parameters (for example, by assigning a value of `OtherAttr` outside the range $[1, 3]$); in this case, the record will still be inserted into `Users`, but will not appear in `OneThree`.

A `SELECT` statement can also contain mathematical operations on the attributes. The operations will not change the record, but only the value returned by the query:

```
SELECT FirstName, OtherAttr*2, POWER(BadgeNum,2)
FROM Users
WHERE OtherAttr > 3);
```

A number of other operators are defined in SQL to refine queries by manipulating the results:

- `COUNT(attr)` returns the number of records associated with a given query; by specifying a single attribute as the argument, it will return the number of non-`NULL` values associated with that attribute.
- `MAX(attr)` returns the largest value among a set of values.
- `MIN(attr)` returns the smallest value among a set of values.
- `SUM(attr)` returns the sum of a set of values.
- `AVG(attr)` returns the average of a set of values.

For convenience and abbreviation, it is possible to define **aliases** either for attributes or for tables. For example:

```
SELECT DISTINCT FirstName AS UniqueName
FROM Users;
```

In practice, it is possible to omit the `AS` keyword and simply write the alias name directly after the attribute or the table:

```
SELECT FirstName NamesEndingI
FROM Users U
WHERE FirstName LIKE '%i';
```

It is possible to group rows that share the same value for a given attribute by using the `GROUP BY` statement. For example, let's imagine we have a table of shipments called `Ships`, where the primary key is the shipment code, stored in `Code`, the countries of origin are stored in `FromCountry`, and the departure cities are stored in `FromCity`.

We want to determine which countries have the largest number of different cities from which shipments depart. The table is the following:

```
SELECT * FROM Ships;
```

Code	FromCountry	FromCity	Rate
001	Italy	Rome	4.7
002	Italy	Venice	3.1
003	France	Nice	3.8
004	Italy	Rome	2.6

We can use the following syntax:

```
SELECT FromCountry, COUNT(DISTINCT FromCity) TotalCities
FROM Ships
GROUP BY FromCountry
ORDER BY TotalCities DESC;
```

FromCountry	TotalCities
Italy	2
France	1

To filter the results of a grouped attribute using functions such as `COUNT`, `SUM`, and `AVG`, it is possible to use the `HAVING` statement:

```
SELECT FromCounty, COUNT(DISTINCT FromCity) TotalCities
FROM Ships
GROUP BY FromCountry
HAVING TotalCities>1
ORDER BY TotalCities DESC;
```

FromCountry	TotalCities
Italy	2

The last feature we will see with `SELECT` is the **join**, which allows the visualization of tables combined through matching values in specific attributes, typically primary keys, that are equal in both tables.

Let's consider another table, called `Orders`, whose primary key is analogous to that of the `Ships` table. This table contains the name of the person who placed the order (`Name`) and their age (`Age`):

```
SELECT * FROM Orders;
```

Code	Name	Age
002	Lewis	39
003	Charles	26
005	Sebastian	36

The first possibility is an `INNER JOIN`, in which the attributes requested by the user are shown only where the values of the attributes in common are equal:

```
SELECT s.FromCountry, o.Name, s.Rate
FROM Ships s
INNER JOIN Orders o ON s.Code = o.Code;
```

FromCountry	Name	Rate
Italy	Lewis	3.1
France	Charles	3.8

We observe a new syntax not seen before: the attributes, in fact, are written in the form `TableName.AttributeName`. This is useful in cases where two tables share the same name for an attribute, as is the case here with `Code`.

Since the common codes between the two tables are only `002` and `003`, only the attributes corresponding to these two records will be shown. We also add that the `INNER JOIN` is — so to speak — *commutative*, in the sense that inverting `Ships s` and `Orders o` between the second and third lines of code would produce the same result.

Two other possible types of join are `LEFT JOIN` and `RIGHT JOIN`, which *are not commutative*. Let's see what happens compared to the inner join:

```
SELECT s.FromCountry, o.Name, s.Rate
FROM Ships s
LEFT JOIN Orders o ON s.Code = o.Code;
```

FromCountry	Name	Rate
Italy	NULL	4.7
Italy	Lewis	3.1
France	Charles	3.8
Italy	NULL	2.6

All records from the `Ships` table are thus taken, and the attributes related to records not present in `Orders` are filled with `NULL` values. This happens because, according to what we have written, it is the `Orders` table that is joined to the `Ships` table; therefore, `Ships` is the table being selected, while `Orders` is added afterward. By inverting the order of the tables, the result will indeed be different:

```
SELECT s.FromCountry, o.Name, s.Rate
From Orders o
LEFT JOIN Ships s ON s.Code = o.Code;
```

FromCountry	Name	Rate
Italy	Lewis	3.1
France	Charles	3.8
NULL	Sebastian	NULL

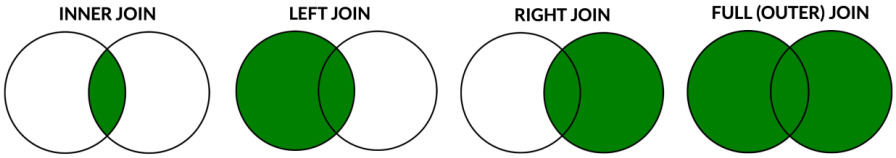
With the `RIGHT JOIN`, we have the opposite behavior: in this case, it is the first table mentioned that is added to the second one. The last possible type of join is the `FULL JOIN`, also called `OUTER JOIN`, where all records are included and any missing attributes are filled with `NULL` values. In SQL, there is no explicit function that directly performs a full join, but it is possible to achieve it by combining a `LEFT JOIN` and a `RIGHT JOIN` through the `UNION` operator.

```
SELECT s.FromCountry, o.Name, s.Rate
From Ships s
LEFT JOIN Orders o ON s.Code = o.Code;

UNION

SELECT s.FromCountry, o.Name, s.Rate
From Ships s
RIGHT JOIN Orders o ON s.Code = o.Code;
```

FromCountry	Name	Rate
Italy	NULL	4.7
Italy	Lewis	3.1
France	Charles	3.8
Italy	NULL	2.6
NULL	Sebastian	NULL



(a) (b) (c) (d)

Figure 5.6: Types of join processes.

Data Control

The last component to analyze is the DCL, or *Data Control Language*, which manages access to the database, user permissions, and data security. We will not go into as much detail as we did for DDL and DML, because DCL is less oriented towards the user experience and more focused on administrative aspects. Therefore, we will only discuss the most important functions.

The starting point is to create a user name associated with a password:

```
CREATE USER 'new_user'@'localhost'
IDENTIFIED BY 'new_password';
```

Through this statement, a new user called `new_user` is created, who can connect exclusively from the local machine (`localhost`), with the associated password `new_password`.

It is possible to assign privileges through the `GRANT` statement and remove them through the `REVOKE` statement. For example, we can grant or revoke the ability to use commands such as `SELECT` and `UPDATE`. It is also possible to revoke more specific privileges, such as the ability to set a field to NULL.

```
GRANT SELECT, UPDATE ON Orders.Name TO 'new_user'@'localhost';

REVOKE UPDATE ON Orders.Name TO 'new_user'@'localhost'
```

Chapter 6

NoSQL

6.1 NoSQL databases

The SQL language fits perfectly with relational databases, which remained predominant until the 1990s, when the amount of data generated by the explosion of the internet and web applications began to grow exponentially.

In 1998, the computer scientist Carlo Strozzi coined the term *NoSQL* when he developed a database management system that did not require the use of the SQL language, promoting the idea of open-source databases that moved away from the traditional relational model. According to Strozzi, the relational model could sometimes represent a constraint, and alternative approaches might be better suited in certain cases.

6.2 Databases partition

An important obstacle in the management of a relational database arises as the number of records increases, making it necessary to **partition** the database — that is, to divide it into smaller parts. As we know from data storage techniques, this leads to the choice between vertical partitioning and horizontal partitioning.

In this context, vertical partitioning means that all records are stored on each disk of the system, with attributes divided across different disks. Such a partitioning scheme does not present a problem for the relational model, since the various partitions can simply be viewed as separate tables sharing the same primary key.

The real challenge arises with horizontal partitioning, also known as **sharding**, where records themselves are distributed across different disks, keeping all attributes of each record together. This situation is not foreseen by traditional RDBMSs, which require each table to be fully contained within a single memory element. To address this, new types of databases must be designed to keep track of the distribution of records.

The idea is to focus on the primary key, since it is the attribute that uniquely identifies each record. Typically, a hash function is applied to the primary key, returning a number; this number is then divided by the number of available partitions P, and the remainder — which will be a number between 0 and $P - 1$ — indicates the index of the disk where the record associated with that key should be stored.

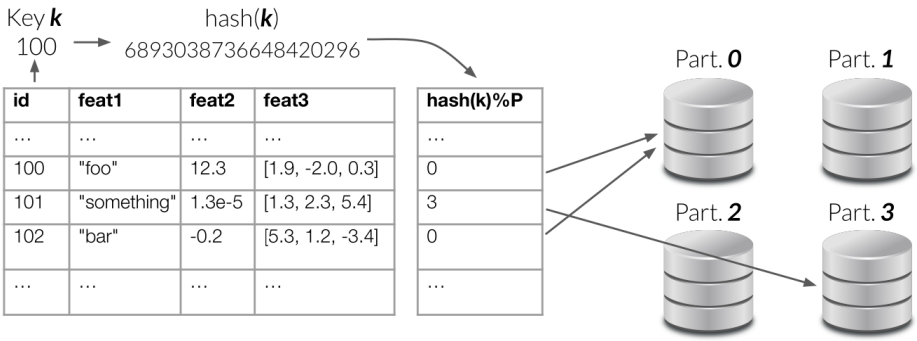
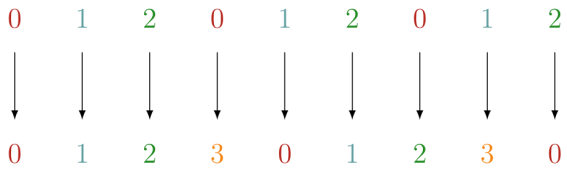


Figure 6.1: Sharding based on hashing and integer division, with $P = 4$ partitions.

However, this method does not scale well: adding a new disk means moving from P to $P + 1$, which requires re-executing all integer divisions and relocating all records according to the new indexing.

In practice, a technique called **consistent hashing** is used instead. This method maps the digests onto a ring of values ranging from 0 to the maximum possible digest value — for example, 2^{256} in the case of SHA-256.

In the previous example, the function used to map digests would have been the modulo operation, `mod(P)`, which returns the remainder of the integer division by P. Let's now see what happens when moving from P to $P + 1$, for instance with $P = 3$:



Since it would be impractical to remap almost all records across the various disks, a different approach is used: each disk is associated with a *sector* of the ring, so that each digest on the ring is *rounded up* to the nearest larger value within the sector, which determines the partition to which it is assigned.

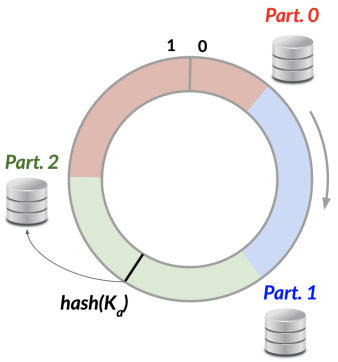


Figure 6.2: Consistent hashing between three disks.

When a new disk is introduced, it will occupy part of a sector that was previously managed by another disk. For example, if the new disk is positioned between partition 2 and partition 0, as shown in Figure 6.2, it would not be necessary to remap the entire database.

Instead, only a portion of the records from partition 0 would need to be transferred to the new partition 3.

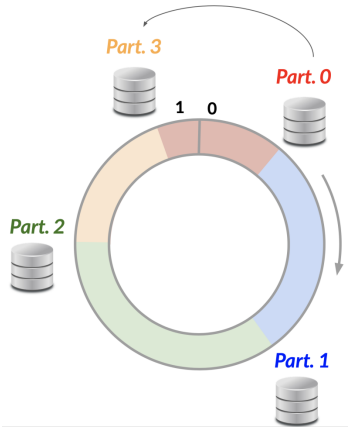


Figure 6.3: Introduction of a new partition with consequent record transfer.

Consistent hashing also lends itself well to data replication across multiple partitions, in order to maintain multiple copies and increase the security of the dataset. Typically, the user decides how many partitions to distribute the copies across, based on the amount of storage space they are willing to sacrifice. Once the number n of partitions is chosen, the copies are placed on the $n - 1$ partitions immediately following the one where the record was originally stored.

6.3 Characteristics of NoSQL databases

The main points that distinguish traditional databases from NoSQL databases are four:

1. **Flexible data models** that can move away from the structured nature of relational databases, offering support for the storage and management of semi-structured and unstructured data.

2. **Horizontal scalability**, as previously discussed, allowing the distribution of databases across multiple nodes of a cluster without significant performance degradation.

3. **Simplified queries** through languages that avoid the use of complex operations like `JOIN`, thus favoring faster response times (especially in distributed environments) over the expressive power of traditional SQL.

4. **Weaker consistency models**, which consequently adhere less strictly to ACID properties, in favor of the scalability and *availability* needed in distributed systems.

The transaction model followed in most NoSQL databases is called **BASE**, named to emphasize its opposition to ACID, similarly to how opposites are defined in chemistry:

- **Basically Available**: individual nodes in a network are generally accessible *almost* all the time, unlike the ACID model where access to the service could be blocked due to a read or write operation in progress, because of *isolation* requirements.

- **Soft state**: the state of the database is not rigidly *persistent*, and may change over time without explicit user intervention — for example, due to data replication across different nodes of a cluster.
- **Eventually consistent**: related to soft state, it means that *consistency*[1] is guaranteed only after some time has passed, and not necessarily immediately after a transaction is completed.

It is important to specify that this last point does not imply that all NoSQL databases are purely AP, since it is not a strict dichotomy where one property is chosen and the other is completely excluded. Both availability A and consistency C are guaranteed along a 'gradient', ranging from stricter conditions (*high availability*) to more relaxed conditions (*eventual availability*). This flexibility allows a single DBMS to adapt to different services and use cases.

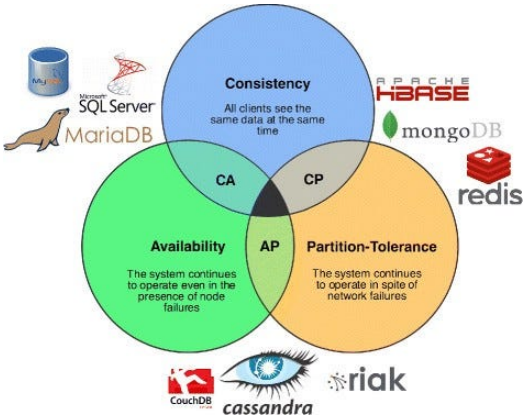


Figure 6.4

6.4 NoSQL databases examples

Key-value databases

The simplest method for storing data is used: each record is associated with a key, similar to how it is done in Python with dictionaries. As in dictionaries, each key can be associated with different types of values, such as a number, a character string, a list of values, and so on. Essentially, this type of storage is *schema-less*, meaning that there is no need to include metadata describing the structure of the data.

The absence of a schema allows for easy and fast distribution across the nodes of a network. Moreover, the key-value model is particularly well-suited for datasets that require a high number of rapid read and write operations. For this reason, the data is often stored directly in RAM to significantly boost performance. Databases that manage data stored in main memory are called **in-memory databases**.

Wide-column databases

Wide-column databases can be seen as an evolution of key-value stores, where the value associated with a key is not a simple item, but a set of columns grouped together into so-

[1] Probably one from CAP theorem, not from ACID model.

called column families. This model is particularly suitable for datasets with large and sparse structures, where the number and types of fields may vary significantly across records.

Each row has a unique key and contains one or more dynamic column families, each of which stores key-value pairs. Wide-column databases are optimized for efficient storage, retrieval, and aggregation of large volumes of data across distributed systems.

Notable examples include Google Bigtable, Apache Cassandra, and HBase.

Document databases

Instead of storing key-value pairs, document databases store *documents* composed of field-value pairs. These documents are essentially information encoded in JSON format, which perfectly fits semi-structured data, where different documents can also be organized into *collections*. Each document is typically identified by a field — usually called `id` — that uniquely identifies the document itself, similar to the role of primary keys in RDBs. Queries are executed by using the fields within the documents.

Time series databases

They are designed to store data indexed by timestamps and to organize them based on time windows. They are particularly well-suited for managing empirical data generated by sensors and provide functionalities for calculating metrics and statistics that help identify trends and patterns.

FullText Search Engines

They are suited for managing unstructured data collected as textual documents. The queries are focused specifically on the text and — similar to what search engines like Google do — return a *ranking* of documents based on matches with the keywords specified by the user.

Graph databases

Graph databases are designed to store and manage data structured as networks of relationships. The fundamental elements are nodes (entities) and edges (relationships between entities), often enriched with properties.

This model is particularly powerful for applications where connections between records are as important as the records themselves, such as social networks, recommendation systems, fraud detection, and knowledge graphs. Queries typically involve traversing paths between nodes and exploring connections. Common languages for querying graph databases include `Cypher` (used by Neo4j) and `Gremlin`.

Notable examples include Neo4j, Amazon Neptune, and ArangoDB.

Vector databases

Vector databases are a recent innovation, designed to efficiently store and retrieve high-dimensional vectors, such as embeddings produced by machine learning models. They are particularly useful for similarity search tasks, such as image retrieval, document similarity, and AI-based recommendation systems.

Examples of vector databases include Pinecone, Milvus, and FAISS.

Chapter 7

Intro to Data Processing

7.1 Processing

Processing is referred as a set of operations to be applied on a dataset to extract higher-level information, including analytics, data reduction, data cleansing, feature enrichment, etc.

More often than not, multiple processing steps have to be performed on the input datasets, including the storage of the intermediate results.

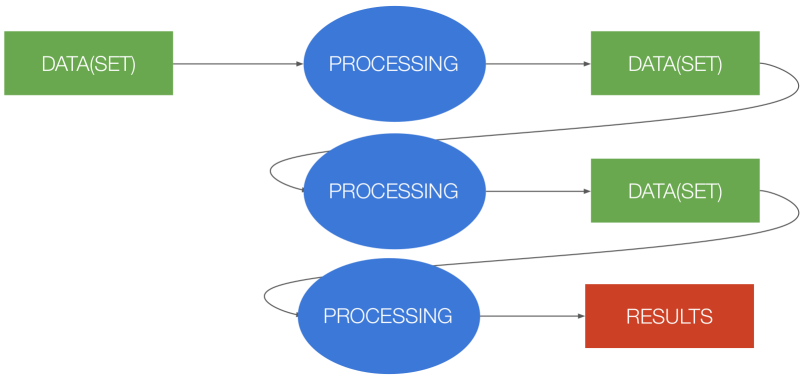


Figure 7.1

Question: How does the processing scale with the data size or task complexity?

At his very core, processing can be seen as the sequential execution of a set of instructions by the CPU, where both the data and the instructions are kept in memory.

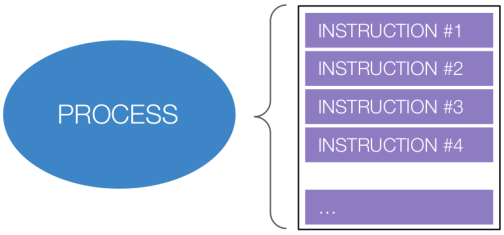


Figure 7.2

The CPU follows a fetch-decode-execute cycle for each instruction:

- **Fetch**: retrieving instruction from memory
- **Decode**: interpreting of the instruction
- **Execute**: execution of the instruction

7.1.1 Processing time

The **CPU processing time** is defined as the time needed to execute the whole set of instructions. This depends on:

- the complexity of the instruction set
- the CPU clock frequency
- the overheads induced by the communication with register and memory

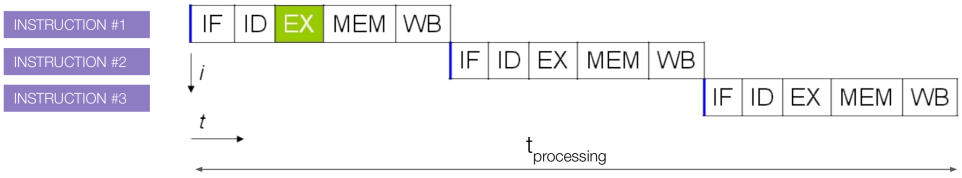


Figure 7.3: Instruction Fetch, Instruction Decode, Execute, Memory Access, Write Back.

How can we improve processing performance? That is, how can we reduce the execution time of a given process? An extremely simplified view of processing time can be obtained by separating it in two components:

- **Wall time** (or response time, or elapsed time) is the total latency to perform a task, including all IO operations (memory and storage access), communication over network, OS overhead, ...
- **CPU time** is the latency due *only* to the instruction processing by the CPU, excluding the time waited performing other activities.

Then, we have

$$t_{\text{CPU}} = \sum_i \frac{(\text{IC}_i \cdot \text{CPI}_i)}{\text{CF}} \tag{7.1}$$

where IC is the instruction count, that is the number of instructions per program, CPI the clock per instruction, that is the number of clock cycles per instruction and CF is the clock frequency, 1/time per clock cycle.

7.1.2 Power wall

An easy solution to boost performance can be increasing the CF; however, today the chips have reached a technological limitation due to the power dissipation

$$P \propto CV^2 f \tag{7.2}$$

where n is the number of transistors, C the transistor capacity, V the transistor voltage and f the clock frequency. This limitation is known as *power wall*.

7.1.3 Pipelining

As performance boosts cannot rely solely on increasing clock frequency, an alternative strategy is to increase the number of instructions a chip can run per second. This is a shift in computer architectures. Then, a speedup is commonly obtained in CPUs with the **Instruction Pipelining**, allowing multiple instructions to be run concurrently on the same ALU. Specifically, instruction pipelining improves execution speed by dividing the processing of a single instruction into multiple sequential stages. This subdivision enables the simultaneous execution of several instructions, each occupying a different stage of the pipeline

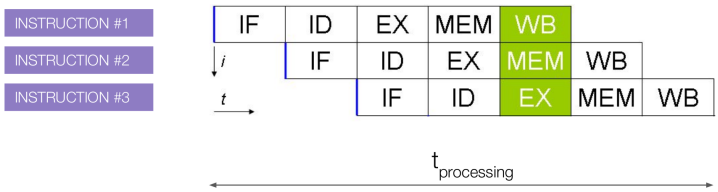


Figure 7.4

In order to achieve better performance, a change of paradigm is required.

7.2 Multi-processing architectures

This change is done by using multiple PUs per CPU (**multi-core CPU**) and/or using multiple CPUs per machine. This can be realized in a variety of ways:

- Symmetric MultiProcessing → Homogeneous CPUs architecture
- Asymmetric MultiProcessing → Heterogeneous CPUs architecture
- Master/Slave → Master CPUs in charge of assigning tasks to the others

With command `lscpu` we can see a lot of infos about CPU. The architecture, for example x86_64 gives the CPU architecture, that is the instruction set. The socket is the number of CPU sockets in the motherboard. The core per socket is the number of physical PUs (cores) in each CPU. The thread per core gives the number of logical PUs per core. These are two independent feeding lanes per CPU: fetch and decode two instructions but can execute only one at a time.

The total number of PUs as seen by the OS is

$$\#\text{CPUs} = \#\text{sockets} \times \#\text{cores per socket} \times \#\text{threads per core} \tag{7.3}$$

With `htop` command we can display the running processes and monitor the system resources.

Multi-processing machines are now common, how can we leverage on them?

7.3 Parallelism

Parallelism refers to the idea of exploiting multi-processing architectures in order to solve complex problems.

There are two kind of parallelism, not mutually exclusive; on the contrary they are often used together:

1. **Data-level parallelism** (DLP): many data items that can be operated at the same time → i.e. split the data and run a task on all elements in parallel.
2. **Task-level parallelism** (TLP): multiple sub-tasks (or **threads**) of a process that can operate independently and largely in parallel. → i.e. split the task into sub-tasks and run them in parallel.

7.3.1 Parallel processing

Parallel processing is the simultaneous execution of a single job by subdividing it in n sub-tasks, executed across m CPUs/cores. The main goal is to reduce the computing time (speedup) of the job.

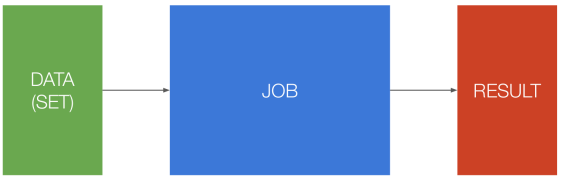


Figure 7.5

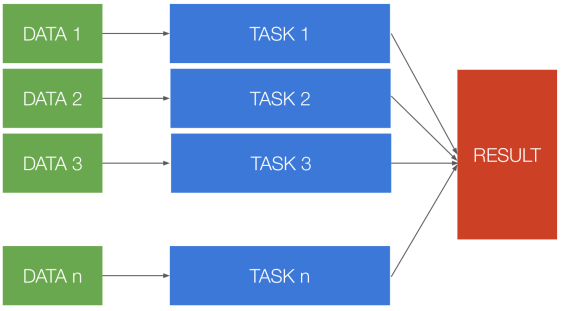


Figure 7.6

Subtasks could be either identical or very different, depending on the type of parallelism and computing architecture.

7.3.2 Flynn's taxonomy

The Flynn's taxonomy is a classification of computer architectures developed in the mid-60s based on the number of

- simultaneously-executed instructions
- parallel data streams processed

SISD

- one instruction on one data stream at a time
- no parallelism at all
- e.g.: 1 single-threaded process in execution on single-core PUs

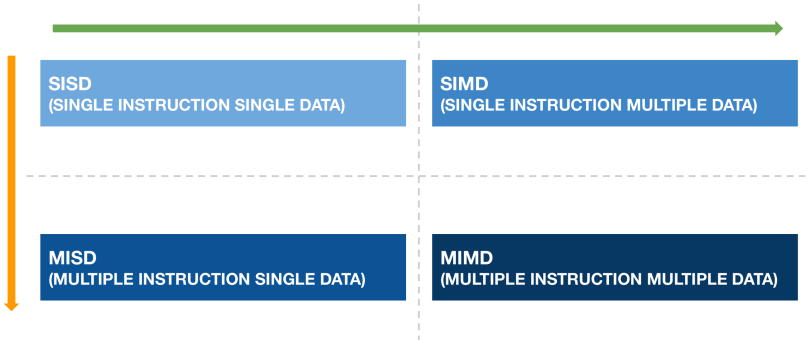


Figure 7.7

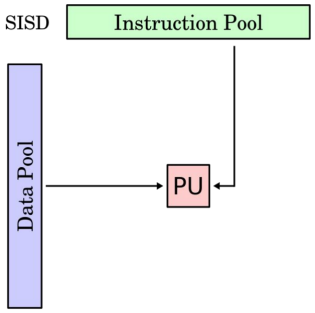


Figure 7.8

SIMD

- one instruction performed on multiple data streams at the same time
- enable data-parallelism (DLP)
- instructions can be further pipelined
- e.g.: specific instructions (avx/mmx/sse) of modern CPUs, most GPUs, specific vector super computer

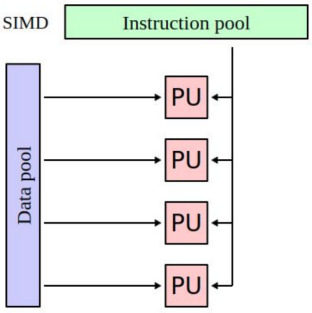


Figure 7.9

MISD

- multiple instructions performed on a single data stream
- very unusual in common data processing, but dedicated to special applications

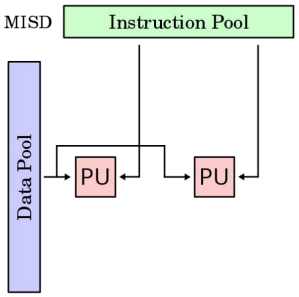


Figure 7.10

MIMD

- multiple instructions performed on multiple data streams at the same time
- flexible parallelism (both DLP and TLP)
- each PU runs independently and (possibly) "asynchronously" from the others

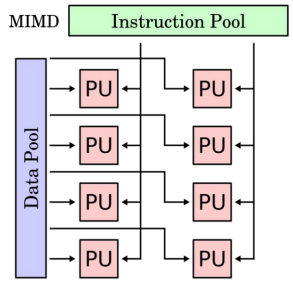


Figure 7.11

A sub-category of MIMD is SPMD → single program multiple data, widely used to run the same entire program (not just single instructions or threads) on multiple "chunks" of data using several PUs → embarrassingly parallel problems.

7.3.3 Speedup in parallel programming

Is it possible to speedup the computation time indefinitely by adding more PUs?

$$1 \text{ PU} \Rightarrow \text{no parallelization}$$

$$n \text{ PUs} \Rightarrow \text{speedup} = \frac{\text{time}_{1\text{PU}}}{\text{time}_{n\text{PUs}}} = \frac{1}{(1-p) + p/n}$$

The last equation is called **Amdhal's law**: p is the fraction of parallelizable processing, $(1-p)$ the fraction of serial processing, n the number of processing units.

For $n \to \infty$ Pus the speedup tends to $\frac{1}{1-p}$.

7.3.4 Amdhal's law

Amdhal's law states that, regardless the number of PUs available for parallel execution, the performance improvement is fundamentally constrained by the serial fraction of the task. In other words, the maximum speed-up achievable through parallelization is limited by the portion of the process that cannot be parallelized. Even with an infinite number of cores, the theoretical upper bound for speed-up is given by $1/(1-p)$. Then, we need to re-think and optimize the process to maximize its parallel fraction.

Amdhal's law is often considered pessimistic because it's based on the assumption that as a problem increases in size, its serial fraction increases proportionally.

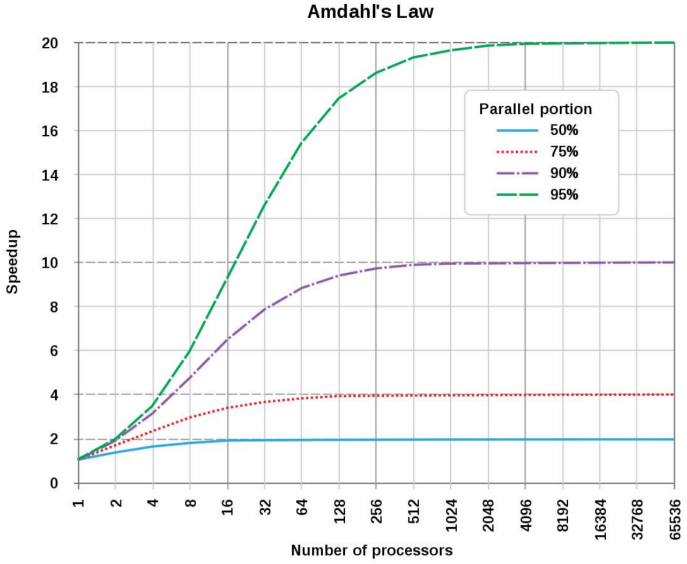


Figure 7.12

7.3.5 Gustafson's law

Gustafson's law offers an alternative perspective on the benefits of parallel computing. Gustafson's approach considers that the pro *problem size* often scales with the number of available processors, thus the parallel portion of the workload grows proportionally, while the serial component remains constant or becomes relatively negligible.

The law is

$$\text{Gustavson's scaled speedup} = 1 + (n-1)p$$

where n is the number of processing units (cores) and p is the parallel fraction of the computation. This implies that, with increasing problem size and a large parallel fraction, the speedup scales almost linearly with the number of processors.

Either ways, **heterogeneous computing** (integrating CPUs + GPUs + ...) aims to get the best of both worlds by speeding up both the serial and parallel components.

Chapter 8

Intro to Concurrency

8.1 Processes and threads

A **process** is an instance of a program in execution. Every process is an independent entity with executable code and memory. A word processor (e.g. Word) is a program → executable file containing the list of instructions. A word processor actively being executed is a process (program ≠ process). Multiple instances of the same program can be executed → same program, but multiple processes.

A **thread** is a basic unit of CPU utilization. Each thread has its own program counter, memory, and a set of registers, but shares code, data, and open files with the other threads of the same process. A thread is part of a process, and a process may contain multiple threads that execute concurrently.

In other words, a thread represents a subdivision of a process that can run independently and in parallel with other threads of the same process. Threads can be executed on a single-core processor (monothreading) or on a multi-core (multithreading).

In a word processor, a thread may check spelling and grammar while another thread processes user input (keystroke), while yet a third thread loads an image from the hard drive, and a fourth does periodic automatic backups of the file being edited.

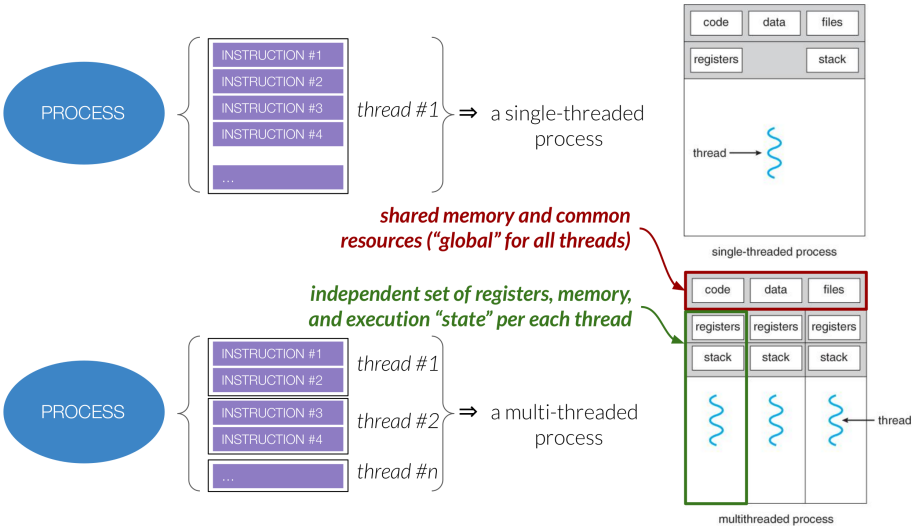


Figure 8.1

8.2 Concurrency and parallelism

Concurrency refers to the ability of a system to handle multiple tasks in *progress* at the same time. These tasks can start, run, and complete in overlapping time periods but not necessarily simultaneously. Concurrency is a way to structure programs to manage multiple operations that can be interleaved. It is particularly useful for I/O-bound tasks, and can occur even on a single-core processor through context switching.

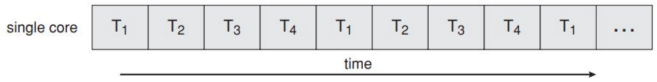


Figure 8.2

- Example: A single-core processor using coroutines or `async/await` to manage network requests concurrently. In particular, we'll see that on a single-core CPU, threads take turns executing thanks to a mechanism called *time slicing*, which is managed by the scheduler. The scheduler allocates a small time slice of the CPU to each thread, then quickly switches to the next one. The result appears to be simultaneous execution, but under the hood it is actually sequential and extremely fast.
- Analogy: One waiter serving multiple tables by quickly switching between them.

Parallelism, on the other hand, refers to the actual *simultaneous execution* or *parallelization* of multiple tasks, by utilizing multiple CPU cores or processors.

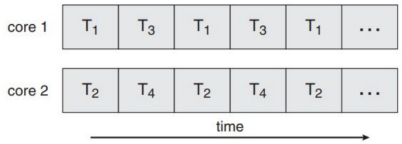


Figure 8.3

- Example: A multi-core CPU executing multiple threads or processes at the same time.
- Analogy: Several waiters serving different tables at the same time.

Key Difference: Concurrency is about *dealing with many things at once* (logical structure), while parallelism is about *doing many things at once* (physical execution).

8.2.1 Multi-tasking

Multi-Tasking is the execution of multiple *concurrent* tasks (processes), made possible by the OS. The OS instructs the CPU to share its time between the various tasks using a scheduling mechanism:

- **Pre-emptive** → the scheduler alternates between the N tasks sharing ∼equally the CPU time.

- **Cooperative** → the system switch processes only upon the active one release control to the scheduler.
- ...

Each task proceeds by executing its instructions sequentially. Therefore, multi-tasking ≠ multi-threading: multi-tasking manages different processes, while multi-threading manages multiple threads within the same process.

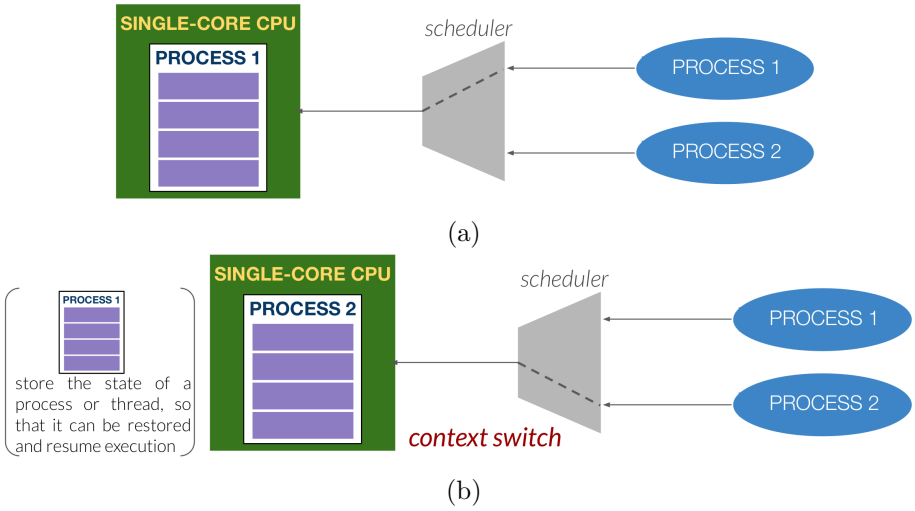


Figure 8.4: On a single-core CPU the scheduler alternates between multiple processes (e.g. process 1 and process 2) by performing a context switch, which involves saving the state of the currently running process and restoring the state of the next one. Although only one process can run at a time on a single-core CPU, this rapid switching creates the illusion of concurrent execution.

8.2.2 Multi-threading

Multi-Threading is the execution of multiple threads within a single process. Depending on the system architecture and OS, thread execution can be either:

- Concurrent, on single-core CPUs using time-sliced multi-tasking;
- Or parallel, on 2+ cores CPUs where threads can run simultaneously on different cores.

Since threads share the same address space, context switching between threads is typically faster and less resource-intensive than switching between separate processes.

Multi-threading allows different components of a program to operate independently while accessing shared global resources such as memory, files, or network connections.

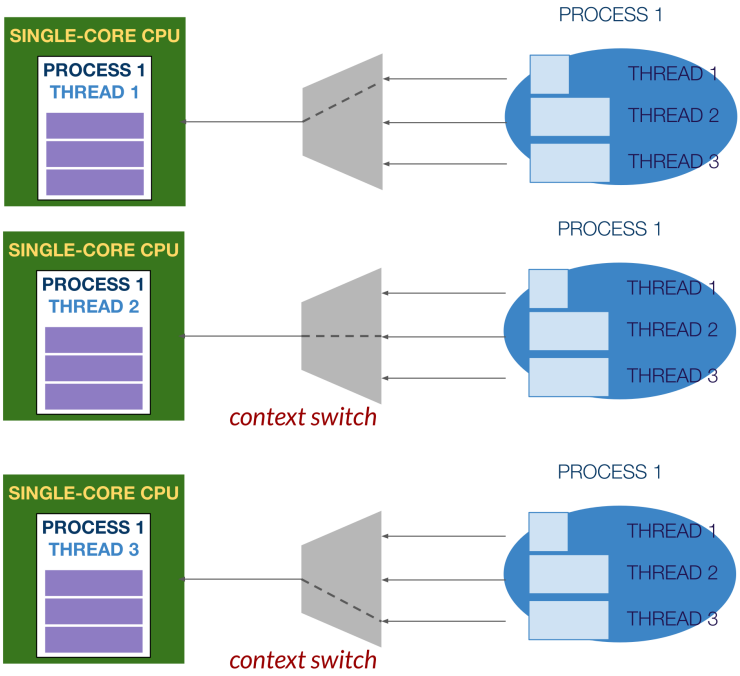


Figure 8.5: In a single-core CPU multi-threading consists in multiple threads belonging to the same process which are executed concurrently through rapid context switching. Although only one thread can be active on the CPU at any given time, the OS's scheduler quickly alternates between them, giving the illusion of simultaneous execution. All threads share the same address space and resources, enabling efficient inter-thread communication and reduced context-switching overhead compared to separate processes.

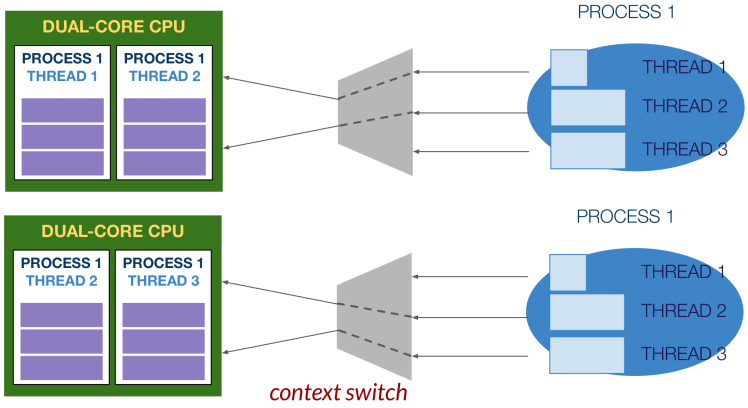


Figure 8.6: In a multi-core CPU multiple threads from the same process can be executed in parallel across different cores. In contrast to single-core systems that rely on time-sliced concurrency, multi-core architectures enable true parallel execution of threads. significantly reducing execution time and improving performance.

8.2.3 Race conditions

Multi-threaded execution can lead to issues if not handled properly. For instance:

- Multiple threads may access the same memory address (e.g. a global variable), since memory is shared among threads.
- The concurrent/simultaneous execution of instructions on shared data can result in inconsistencies.

A **race condition** occurs when two or more threads access a shared resource concurrently, and the final outcome depends on the particular order in which the access takes place.

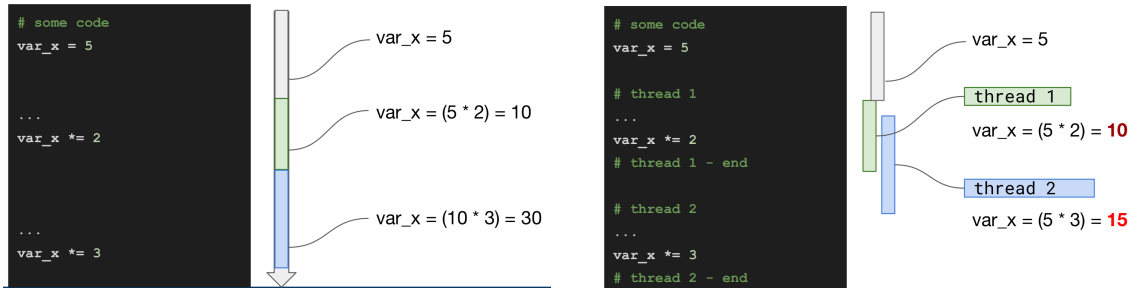


Figure 8.7: Example of a race condition caused by unsynchronized access to a shared variable `var_x` by two threads. In the first scenario (left), the operations are serialized, so that `var_x` is first updated to 10 by thread 1, and then to 30 by thread 2, preserving the correct sequence of operations. In the second scenario (right), both threads read the initial value 5 before either performs the multiplication, resulting in conflicting updates and an incorrect final value (15 instead of 30).

8.2.4 Thread synchronization

In multi-threaded programming, synchronization mechanisms are essential to avoid race conditions and to ensure safe and coordinated access to shared resources. These are secured by means of:

- Lock/Mutexes (mutal exclusion). A mutex (a particular type of lock) is a synchronization mechanism that ensures only one thread at a time can access a specific section of code or a shared resource (like a variable or file). When a thread acquires a mutex, it gains exclusive access. Other threads trying to acquire the same mutex are blocked until the mutex is released, preventing simultaneous access.
- Binary semaphores which are similar to mutex but with no ownership. This is useful for signaling between threads.
- Counting semaphores allow access to a limited number of identical resources.

A **deadlock** is a condition that occurs when two or more threads are waiting indefinitely for resources held by each other, such that no progress can be made.

8.3 GIL: Global Interpreter Lock

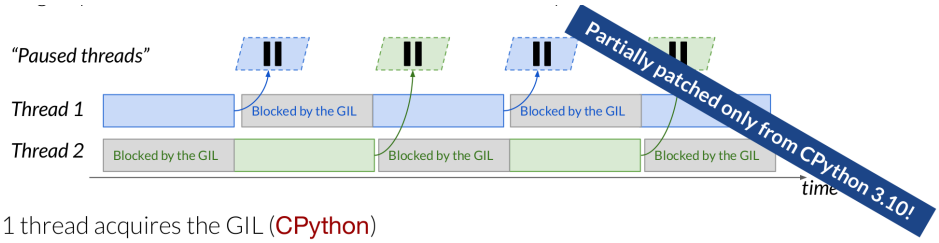
The **Global Interpreter Lock** (GIL) is a mutex that protects access to Python objects in the CPython interpreter (the most widely used implementation of Python). It ensures that only one thread at a time can execute Python bytecode, even on multi-core systems. As a result, multi-threaded Python programs cannot achieve true parallel execution of threads when using CPython.

This limitation effectively disables thread-based parallelism for CPU-bound tasks. While multiple threads can still be created and used (e.g., for I/O-bound operations), only one of them will be allowed to execute Python code at any given moment due to the GIL.

The GIL was introduced to simplify memory management and ensure thread safety within CPython. Python's memory model (based on reference counting) was not designed for concurrent access from multiple threads, so the GIL acts as a global lock around interpreter internals to prevent race conditions and deadlocks. However, this comes at the cost of poor scalability for multi-threaded CPU-bound programs.

It's important to note that the GIL is not a feature of the Python language itself, but rather a limitation of the CPython implementation. Other Python interpreters, such as Jython (based on Java) and IronPython (based on .NET), do not implement a GIL and can support true thread-level parallelism—assuming the underlying platform allows it.

Despite the GIL, race conditions may still occur. The operating system's scheduler is unaware of the GIL and may switch between threads running on different cores. During this context switching, some overlap in execution can occur, especially when threads share resources or call into native code (e.g., C extensions), leading to subtle and hard-to-debug issues.



- 1 thread acquires the GIL (CPython)
- All others threads are scheduled (OS) and make requests, denied by the GIL (CPython)
- The GIL (CPython) is released and context switch happens (OS)

Due to these limitations, multi-threading in Python (under CPython) is generally not suitable for CPU-intensive tasks. Instead, Python developers often prefer the use of the `multiprocessing` module, which spawns separate processes—each with its own Python interpreter and memory space—thus bypassing the GIL and allowing real parallel execution across multiple cores.

8.4 Summary: in the end threads or processes in python?

Threading:

- Threads share the same memory and can write to and read from shared variables
- Due to CPython GIL, two threads won't be executed at the same time, but only concurrently
- Effective for I/O-bound tasks (e.g. data access from/to disk or remote locations)

- Can be easily implemented in python with the `threading` module

Multiprocessing:

- Every process has is own memory space
- Every process can contain one or more sub-processes
- Can be used to achieve parallelism by taking advantage of multi-core machines
- Effective for CPU-bound tasks (e.g. data crunching)
- Can be easily implemented in python with the `multiprocessing` module

Chapter 9

Intro to Distributed Processing

9.1 Parallel programming

As previously discussed, processing can scale with the number of processing units n, provided that the workload is at least partially parallelizable. Also, empirical studies on speed-up typically show that execution time tends to scale reasonably well with n, although the exact behavior depends on the task, the figure of merit and the use case. In fact, a lot of processes are inherently highly scalable, particularly those that are heavily data-parallel, such as in the SPMD paradigm.

We now aim to perform computations on very large datasets, which may also be distributed across different nodes or systems. To accommodate this, we will increase the number of processing units n.

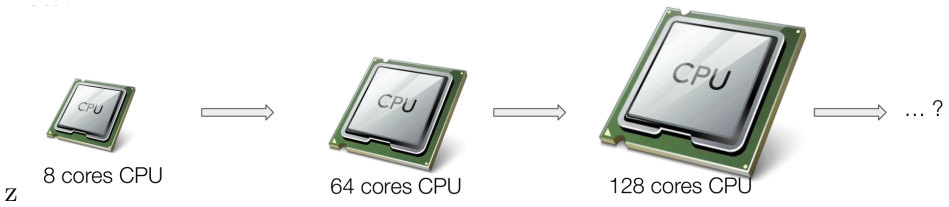


Figure 9.1

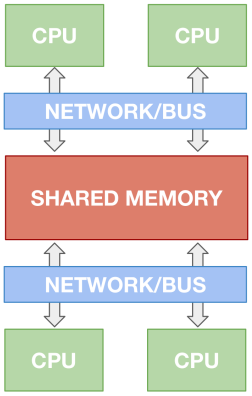
This can be done in two ways: scale up and scale out. Since scaling up is straightforward but limited, we focus on scale out.

9.2 Parallel computing architectures

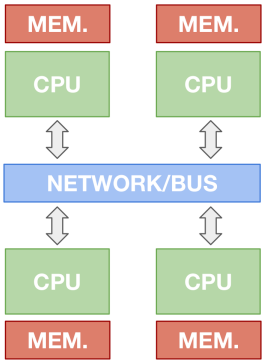
Different parallel computing architectures can be defined depending on how computing resources - particularly memory and processors - are shared. Two common models are:

Shared Memory MultiProcessor (SMP): in a SMP architecture, multiple CPUs access the same physical memory space. All threads and processes can communicate through this shared memory without explicit messaging. SMP is the basis of most modern computers, including desktops, servers and smartphones. Threads within the same process can efficiently share data. However, scaling SMP systems to a large number of processors becomes increasingly challenging and expensive due to memory contention and hardware complexity.

Distributed Memory MultiProcessor (Cluster). In a cluster, each processor (or node) has its own private memory. Nodes communicate via a high-speed network. Clusters can be built using commodity hardware, making them cost-effective for large-scale computations. This model offers excellent scalability, especially for data-parallel tasks. However, it introduces communication overhead and programming complexity, as explicit messaging is required between nodes (e.g. via MPI, Dask, or Spark).



(a) SMP



(b) Cluster

9.3 Distributed computing: cluster

In a cluster, we distribute the processing across multiple *nodes* of a set of computing resources linked together by high-speed networks. Each node is an independent machine equipped with its own CPUs, memory and storage, capable of executing tasks locally. The nodes share computational workload as *vitual massive computer*. From the user's perspective, the cluster is exposed as a single, unified system.

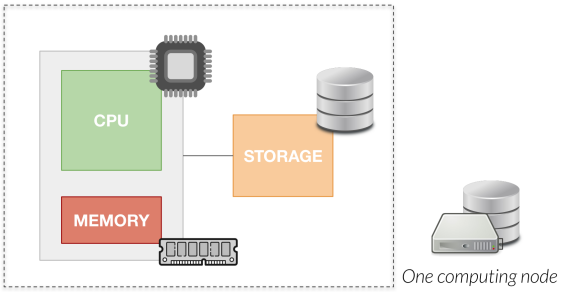


Figure 9.3

Coordination between nodes is handled by a scheduler or resource manager, which distributes tasks, manages job execution and balances the workload. Communication between nodes is achieved through message-passing interfaces (MPI) or distributed computing frameworks (dask or spark).

In particular (Figure 9.4), a **Head Node** (HN) serves as the access point to the cluster for the clients. It is responsible for coordinating and distributing the workload to CNs. The **Computing Nodes** (CNs) are the nodes where the actual processing takes place. Typically,

they are invisible for the users, accessed only via the HN. The **Interactive Nodes** are user-accessible machines that function like standard computers. Used for testing, storage, access, or launching jobs. The **Specialized Nodes** are nodes with dedicated hardware resources designed to handle specific workloads or computationally intensive tasks.

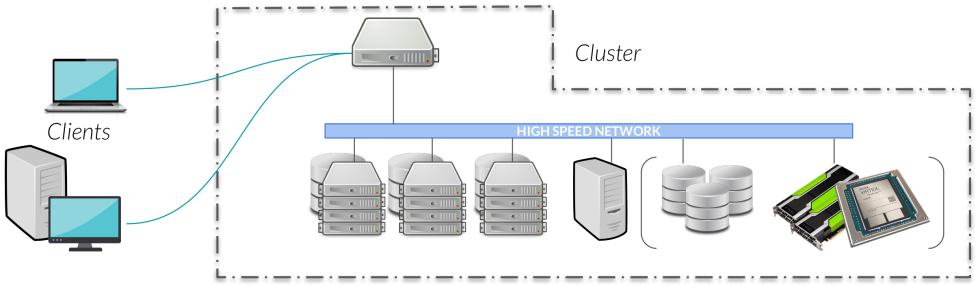


Figure 9.4: On top the Head Node. At the bottom, from the left: Computing Nodes, Interactive Nodes and Specialized Nodes.

Most of today's supercomputers are actually clusters of extremely powerful nodes, often equipped with additional accelerators (GPUs) and hosting a vast number interconnected units. On the other hand, clusters can be an excellent solution to exploit modest HW to efficiently perform parallel tasks.

9.4 A common job submission pattern: batch systems

Users usually schedule and submit jobs to a cluster through a **batch system**, each job including:

1. an executable + optional arguments (e.g. number of CPUs, memory, ...)
2. info on software requirements (e.g. libraries)

Once submitted, the batch system places the job in a queue, waiting for the appropriate resources to become available. The batch system's scheduler manages workload, prioritizes jobs based on factors such as user policies, resource availability, and past usage, and determines the execution order.

When ready, the job is dispatched to the Computing Nodes for execution.

Common batch system schedulers include:

- PBS
- SLURM
- LSF
- HTCondor

Optimizing the execution of complex tasks on parallel and distributed computing systems requires familiarity with specific programming models and frameworks designed to manage concurrency, memory access, and communication. Key technologies include:

- OpenMP (Open Multi-Processing): a shared-memory parallel programming model for multi-core CPUs, typically used on single nodes to parallelize loops and regions using compiler directives.

- MPI (Message Passing Interface): the standard framework for communication and synchronization between processes running on different nodes in a distributed-memory cluster.
- CUDA (Compute Unified Device Architecture) and HIP (Heterogeneous-computing Interface for Portability): programming platforms used to exploit the massive parallelism of GPUs, enabling fine-grained vector and matrix computations.
- Other tools and frameworks such as OpenACC, Dask, Spark, and OpenCL, which address different domains of parallelism, ranging from accelerators to large-scale data processing.

Chapter 10

Map-Reduce and Hadoop

10.1 Big data

Starting from the early 2000s several improvements over the processing of large datasets came from BigData technologies, born and maintained mostly from institutions and companies *outside* of the scientific community.

A commonly adopted framework to describe Big Data is the **3Vs model**, which identifies three fundamental dimensions:

- **Volume**: refers to the sheer quantity of data generated, often measured in kilobytes to terabytes (or more). Data may come from online/offline transactions and is typically stored in structured formats such as records, tables, or files.
- **Velocity**: describes the speed at which data is produced and must be processed. This includes both real-time and batch data streams, originating from online and offline sources.
- **Variety**: captures the heterogeneity of data types, including both structured and unstructured formats. This includes texts, images, videos, and data generated by humans or machines.

Effective Big Data processing relies on a layered architecture that spreads computation and storage across many nodes. This approach makes it possible to process huge datasets by using many resources in parallel. The key ingredients to Big Data processing and analytics are illustrated in Figure 10.1. From the bottom, we have

- The Cluster Nodes, which are the actual physical or virtual machines (servers or cloud instances) that do the computing work. As discussed, each node has its own CPU, memory and disk, and they all work together as part of the same distributed system.
- Then, we have the Distributed FS, which is a shared storage system that spans all the nodes, making files accessible from anywhere in the cluster. For example HDFS.
- The Cluster Resource Manager is the scheduler of the cluster, which decides which tasks run where, and manages how CPU, memory, and other resources are used. Examples: YARN, Kubernetes, Slurm.
- In general, the Distributed Processing framework is the software tool that actually manage the execution of tasks across the cluster. It breaks the job into smaller parts,

run them on different nodes, and take care of errors or delays. Examples: Apache Spark, Flink, Dask.

- Finally, at the top-level abstractions, there are high-level tools or APIs that let users write code without worrying too much about what happens behind the scenes (e.g. Spark SQL or Dask DataFrame).

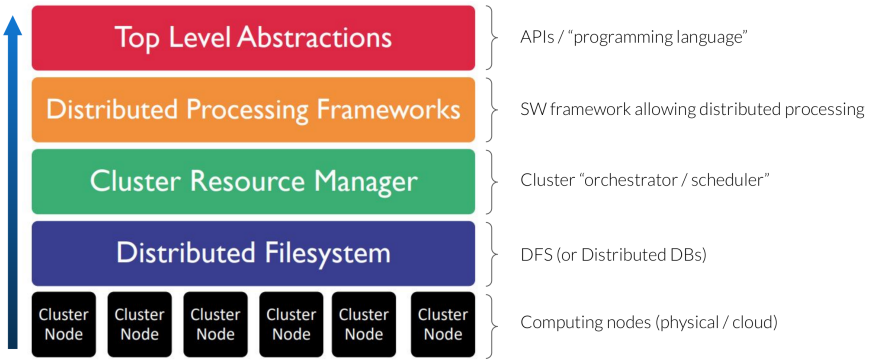


Figure 10.1

10.2 Hadoop

Born in 2006 within Yahoo! and inspired by a 2003 research paper on the Google Distributed File System, Hadoop is an open source framework - managed by the Apache foundation - designed for large-scale distributed data processing. Developed and written in Java, Hadoop is built to scale horizontally, meaning it can efficiently handle increasing amounts of data by simply adding more computing nodes to the cluster

Hadoop allows the management and processing of vast volumes of data across distributed computer clusters. Its main components include:

- Hadoop Common → a collection of Java-based libraries and utilities used by the other Hadoop modules
- Hadoop DFS (HDFS) → a fault-tolerant, high-throughput distributed storage system
- Hadoop YARN → the cluster resource manager and scheduler that allocates CPU, memory and task execution
- Hadoop MapReduce → a programming model for parallel data processing across the cluster

For many years, Hadoop has been the cornerstone of Big Data architectures, serving as a primary tool for storing and processing massive datasets in parallel. While its popularity has declined, it remains relevant in academic environments.

Let's take a closer look at the components that form the Hadoop Ecosystem (Figure 10.2)

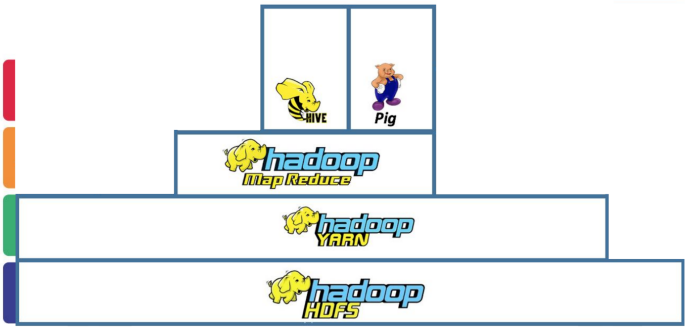


Figure 10.2: Hadoop ecosystem.

10.2.1 HDFS recap

In Figure 10.3 the structure of HDFS is shown. The Client interacts with the system to read/write files. The NameNode stores metadata (file names, block locations, etc), while the Secondary NameNode periodically creates checkpoints to help recover metadata. The actual data is stored across multiple DataNodes, which handle the reading, writing and replication of file blocks. When a file is written, it is split into blocks and distributed across DataNodes with replication. The NameNode coordinates everything, but the Client communicates directly with the DataNodes for reading and writing.

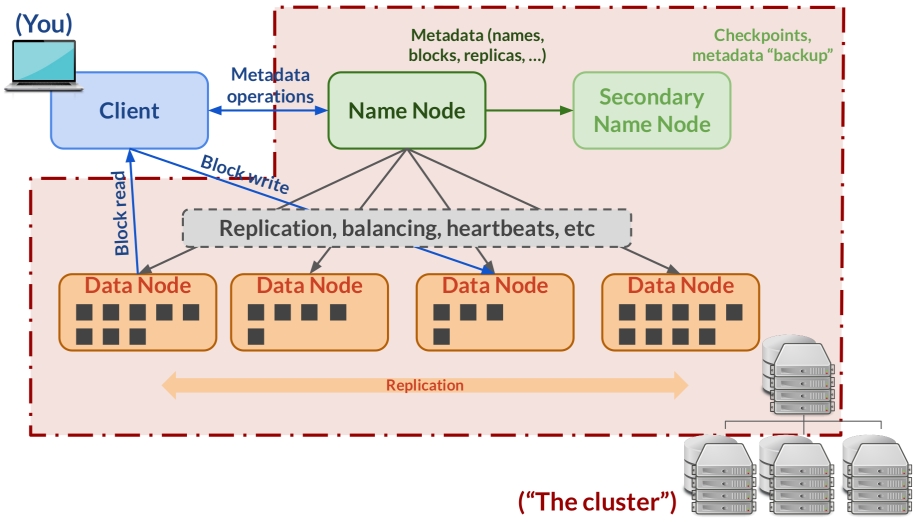


Figure 10.3: HDFS

10.2.2 Map-Reduce in Hadoop

Map-Reduce is a programming paradigm used in Hadoop for distributed processing to elaborate a large quantity of data in a parallel and scalable way. It is not a programming language per se, but rather a conceptual model for building distributed algorithms that can handle massive datasets.

It consists of two main phases:

- **Map** phase: in which the input is split into many sub-tasks, each represented as a

pair of `(key,value)`. Then, a user-defined function is applied independently to each pair, producing a list of intermediate `(key,value)` pairs.

$$(\mathrm{k}_{\mathrm{input}}, \mathrm{v}_{\mathrm{input}}) \xrightarrow{\textbf{map}} \mathrm{list}(\mathrm{k}_{\mathrm{intermediate}}, \mathrm{v}_{\mathrm{intermediate}}) = [(\mathrm{k}^1_{\mathrm{int}}, \mathrm{v}^1_{\mathrm{int}}), (\mathrm{k}^2_{\mathrm{int}}, \mathrm{v}^2_{\mathrm{int}}), (\mathrm{k}^3_{\mathrm{int}}, \mathrm{v}^3_{\mathrm{int}}), \ldots]$$

- **Reduce** phase: the system groups together all intermediate values that share the same key. Then, an aggregation function is applied to those grouped values, producing the final result. In other words, it take as input the list of intermediate `(key,value)` pairs, run aggregation of the values from all pairs with the same key to produce a final list of `(key,value)` pairs, or a single value, depending on the use case.

$$\mathrm{list}(\mathrm{k}_{\mathrm{int}}, \mathrm{v}_{\mathrm{int}}) \rightarrow \mathrm{list}(\mathrm{k}_{\mathrm{int}}, \mathrm{list}(\mathrm{v}_{\mathrm{int}})) \xrightarrow{\textbf{reduce}} \mathrm{list}(\mathrm{k}_{\mathrm{out}}, \mathrm{v}_{\mathrm{out}}) \quad \text{or} \quad \mathrm{v}_{\mathrm{output}}$$

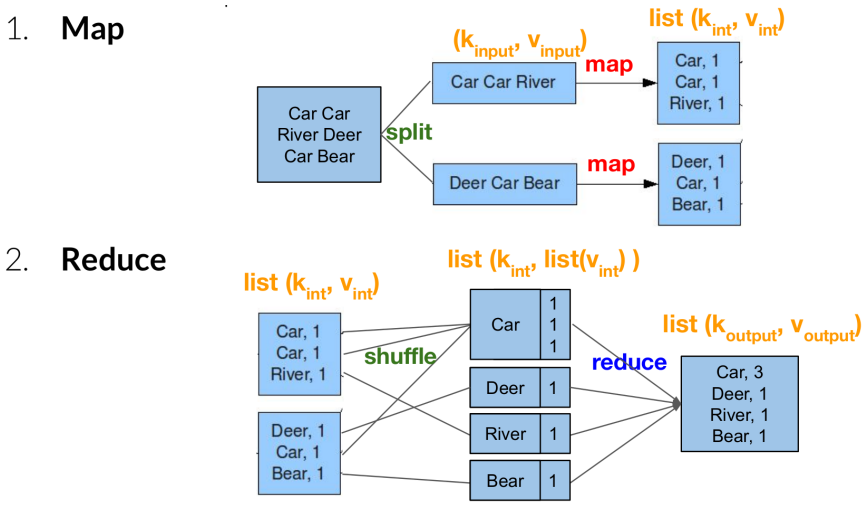


Figure 10.4: Example of the MapReduce programming model applied to a word count task. The input is split and mapped into intermediate key-value pairs (each word with count 1), which are then grouped by key during the shuffle phase. Finally, the reduce phase aggregates the values to compute the total count for each word.

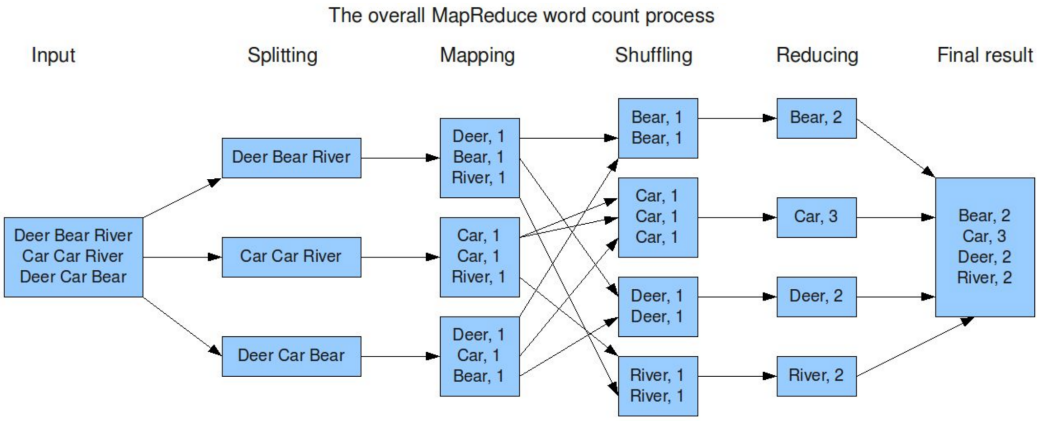
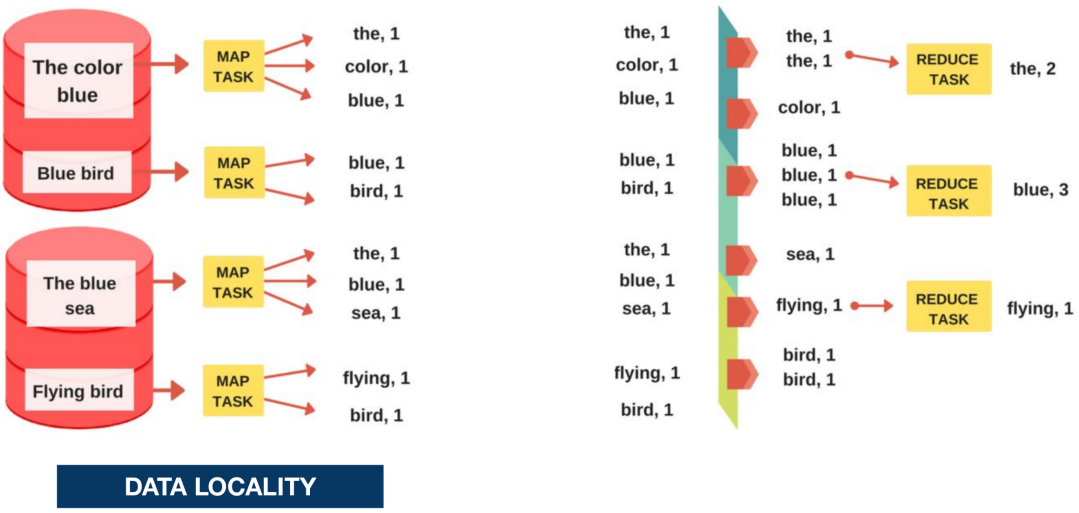


Figure 10.5: Overview of the MapReduce process applied to word count.

Let's see now how MapReduce works in Hadoop. During the map phase (Figure 10.6a), data is read from the distributed file system, where it is already partitioned and stored across multiple nodes of the cluster. Each node processes its own portion of data by applying the map function, which transforms input records into a set of intermediate key-value pairs. The map tasks operate *independently of each other*, and the intermediate results are stored locally on the same node where the map tasks runs - a key feature known as data locally, which reduces data transfer overhead.

Once all map tasks are completed, the reduce phase begins (Figure 10.6b). This phase starts with the **shuffle and sort** step: the intermediate key-value pairs are grouped by key and redistributed (shuffled) across the cluster to the appropriate reduce tasks. Each reduce task then receives all values associated with a given key and applies the reduce function to produce the final aggregated result.



(a) (b)

Figure 10.6

10.2.3 YARN

YARN (Yet Another Resource Negotiator) takes the role of the OS for Hadoop, handling resource management and job scheduling across the cluster. Its main components are:

- **Resource Manager** (RM): it coordinates how cluster resources are assigned to applications. It is composed of:
  - the Scheduler, which decides how resources are distributed based on availability and configured policies;
  - the Applications Manager, which manages the submission and life cycle of applications.
- **Application Master** (AM): it negotiates resources with the RM and coordinates with the NMs to execute and monitor the tasks.
- **Node Manager** (NM): it runs on each node and takes care of launching and monitoring containers, as well as reporting resource usage.

- **Container**: a bundle of resources allocated by the RM (memory and CPU) allocated by the RM to run a specific task

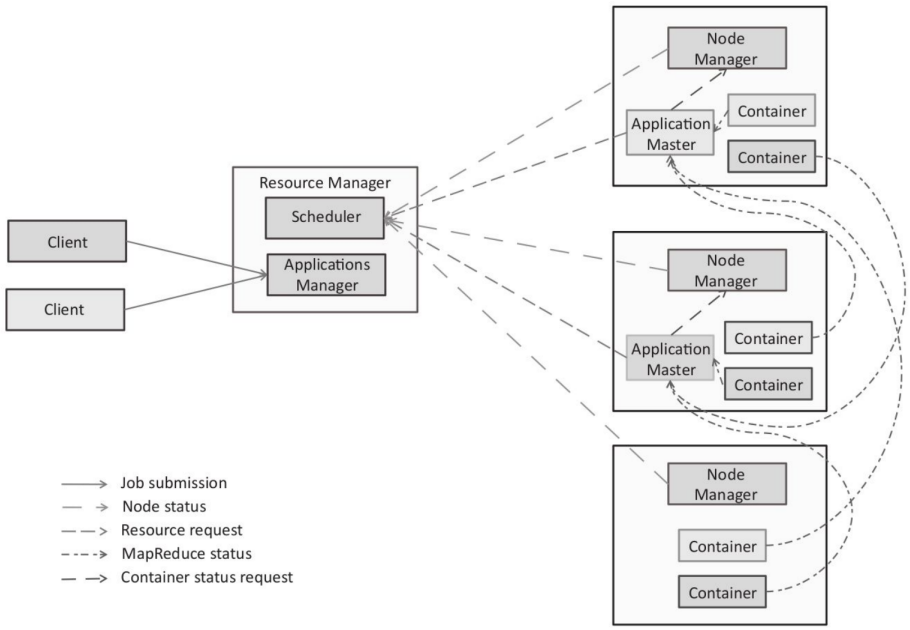


Figure 10.7: YARN

10.2.4 Hadoop high-level abstractions

To simplify writing and managing MapReduce jobs, Hadoop offers several high-level abstractions:

- **Pig** is a high-level language developed at Yahoo! that allows users to write data analysis scripts which are then translated into MapReduce operations.
- **Hive** originally developed at Facebook, provides a SQL-like interface (HiveQL) for querying data stored in HDFS and organized in tables, making it easier for users familiar with relational databases.
- **Oozie** is a workflow scheduler that enables the orchestration of multiple Hadoop jobs, allowing them to be chained and managed as part of complex data pipelines.

Chapter 11

Spark

11.1 Spark vs Hadoop

Spark is an open-source distributed computing framework designed for fast and flexible data processing across clusters. Developed as a response to the limitations of the Hadoop MapReduce paradigm, Spark offers a modern alternative that supports a unified engine for batch processing, real-time streaming, SQL queries, machine learning and graph analytics. Unlike Hadoop, which is based on a rigid two-phase model, Spark allows more expressive data workflows through a high-level API, enabling complex pipelines such as filtering, joining, grouping, and aggregation using intuitive syntax similar to pandas or SQL.

Spark is developed and written in Scala, while it includes also APIs for Java, Python and R languages. However, performance will be optimal only when using Scala. In Figure 11.1 is shown the Spark ecosystem.

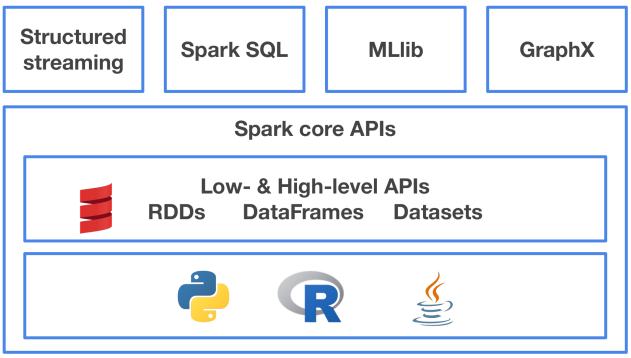


Figure 11.1: Spark ecosystem.

One of Hadoop's main drawbacks lies in its reliance on disk I/O between each processing phase - data must be written to disk after the map step, during shuffling, and again after the reduce step - causing significant performance overhead. Furthermore, Hadoop lacks intelligent in-memory caching requiring repeated reads from HDFS even for iterative operations like those in machine learning, making it inefficient for such tasks. Spark addresses these issues by enabling in-memory computation through resilient distributed datasets (RDDs) and DataFrames, reducing I/O latency and making iterative algorithms feasible and fast.

In addition, Spark does not depend strictly on MapReduce, allowing it to overcome many of the architectural limitations of Hadoop, including its high latency and inflexible job structure. While Hadoop is primarily suited for batch processing of static ("at-rest") data,

Spark supports both batch and real-time streaming workloads within the same ecosystem. From an operational standpoint, Spark also simplifies cluster management by integrating its components more cohesively, whereas Hadoop requires separate configurations for YARN, HDFS, and MapReduce.

11.2 Spark cluster architecture

Spark follows a programming paradigm centered on **Resilient Distributed Datasets** (RDDs). At the heart of its execution model are two key concepts:

- **Transformations**, which are *lazy* operations such as `map()`, `filter()`, `join()`, or `groupBy()`, which do not perform any computation immediately. Instead, they build a **Directed Acyclic Graph** (DAG) representing the logical execution plan. Each transformation produces a new RDD or DataFrame.
- **Actions**, which trigger the actual execution of the computation, such as `collect()`, `count()` or `reduce()`. When an action is called, Spark optimizes and schedules the tasks required to produce the result.

Internally, Spark follows a **master-slave** architecture, and its execution flow involves several components:

1. The **Driver program** is the main process that defines the transformations and actions. It coordinates the Spark application and communicates with the cluster.
2. The **Cluster Manager** (e.g. YARN) allocates CPU and memory resources across the cluster.
3. The **Executors** are worker processes launched on cluster nodes. They are responsible for executing the individual tasks assigned by the Driver and reporting back the results.
4. The **Scheduler**, within the Driver, builds the DAG of execution stages and splits it into tasks that are distributed to the Executors.

When a Spark application is submitted, the Driver creates a SparkContext which serves as entry point to Spark's functionality and handles communication with the Cluster Manager. The Driver then schedules tasks across the available Executors and monitors their execution. Thanks to its DAG-based engine, Spark can optimize execution plans and reuse intermediate results in memory, making it highly efficient for complex and iterative workloads (Figure 11.2)

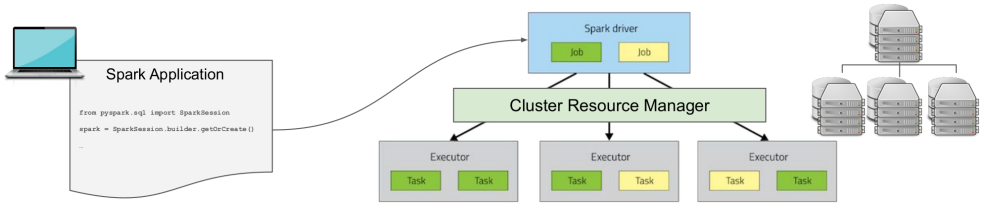


Figure 11.2

Each executor is launched within a Java Virtual Machine on a worker node. Executors cannot span multiple nodes, but a single node may host multiple executors depending on its hardware configuration (e.g. number of cores and memory available) (Figure 11.3)

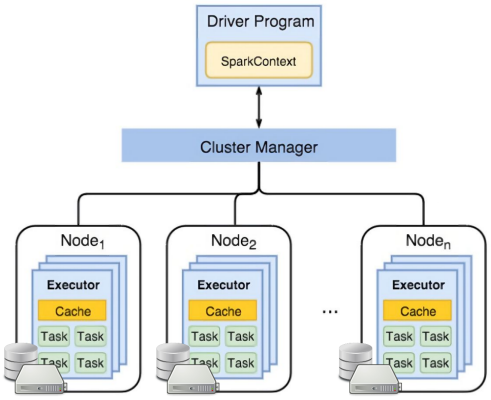


Figure 11.3

11.2.1 RDDs

As we mentioned, RDDs are the core data structure in Spark. They represent **immutable**, **distributed collection of elements** (such as rows, records, or numbers) that can be processed in parallel across a cluster. The main characteristics of RDDs are:

- Immutable: once an RDD is created, it cannot be modified. Any transformations (e.g. `map`, `filter`) produces a new RDD instead of altering the original one.
- Distributed: the data is partitioned and distributed across the executors, taking advantage of data locality to optimize performance.
- Resilient: in case of a node failure, spark can rebuild the lost partitions using RDD's lineage, which keeps track of how the dataset was derived.
- Lazy evalutation: transformations on RDDs are not executed immediately but only when an action (e.g. `collect`, `count`) is called. This allows Spark to optimize the execution plan.
- Unstructured and language-native: data in RDDs is typically stored as collections of Java or Python objects.
- In-memory persistance: Spark can keep RDDs in memory across multiple operations, which makes them especially useful for iterative algorithms or repeated access.

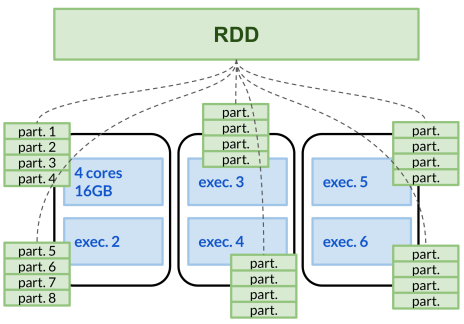


Figure 11.4

11.2.2 Transformations and actions

Transformations are operations that define a "new" RDD based on an existing one. A key concept in understanding transformations is the type of dependency they create between the resulting RDD and its parent(s). Dependencies determine how data is moved and processed across partitions:

- **Narrow dependencies** → each partition of the child RDD depends on only one partition of the parent RDD. Examples: `map`, `filter` and `sample`. These transformations have the following properties:
  - Data stays on the same node, promoting data locality.
  - No suffle is required acrossa cluster.
  - They can be executed in a pipeline, allowing for more efficient processing.
- **Wide dependencies** → each partition of the child RDD depends on multiple partitions of the parent RDD. This occurs in transformations like `groupByKey`, `join` or `distinct`. Their characteristic includes:
  - A shuffle phase is needed to redistribute data across hte cluster.
  - They are more expensive, requiring disk and network I/O.
  - Execution must wait for all parent stages to complete, introducing barries and latency.

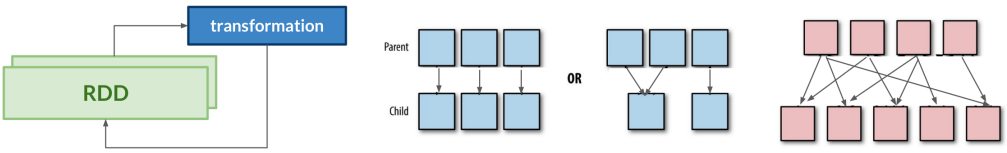


Figure 11.5

On the other hand, **actions** are operations that trigger execution and return a value. Examples: `count`, `collect`, `reduce`. They cause Spark to materialize the DAG of transformations and begin computation across the cluster.

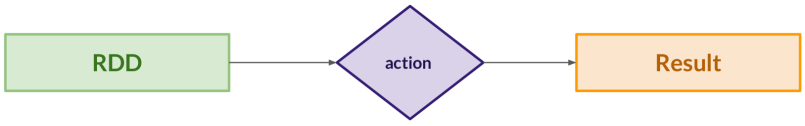


Figure 11.6

As we already mentioned, transformations are **lazy**, meaning that their execution will not start until an action is triggered. Instead of applying the transformation to all the data stored in all partitions at the same time, Spark keeps the transformation logic - called the "recipe" - and maintains metadata about data distribution across nodes. This minimizes communication between the driver and the executors, and allows Spark to optimize execution plans before any computation is launched.

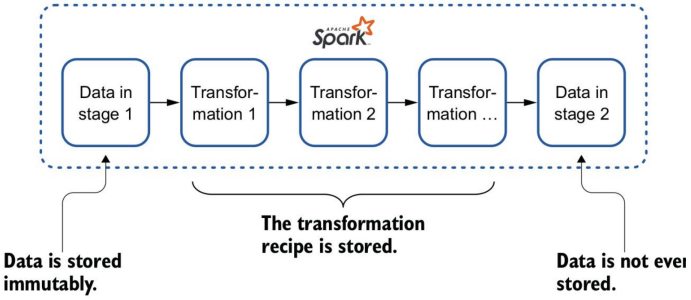


Figure 11.7

11.2.3 DAG scheduler

To optimize the execution of a job, Spark represents all operations as a **Directed Acyclic Graph** (DAG). This graph models the dependencies between the various transformations applied to the data.

The DAG scheduler analyzes the graph and splits it into **stages**, which are groups of operations that can be executed without requiring data exchange between partitions (i.e. narrow transformations). Within a stage, all operations are **pipelined**, meaning they are combined into a single sequence of execution steps, minimizing intermediate results and improving performance.

Each Spark job is thus interpreted as a chain of transformations linked in an acyclic graph. If two or more operations can be executed locally within the same partition, without needing data from other partitions, they belong to the same stage. These are the narrow transformations. Once all the stages are defined, the driver breaks them down into tasks, one per data partition, and dispatches them to the executors, where they are run in parallel.

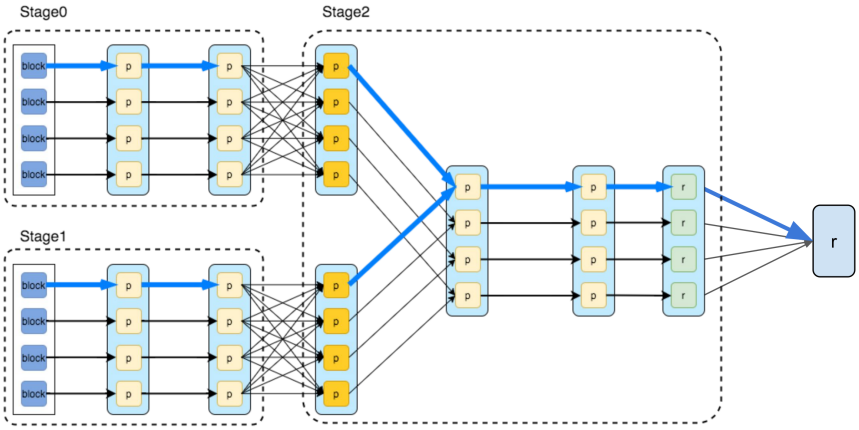


Figure 11.8: Each colored block represents a stage of execution, composed of parallel tasks (P) that proces indivudal data partitions (blocks). The input data is typically read from HDFS. Shuffle operations - shown as crossing arrows between stages - represent data exchanges across partitions on different nodes and define wide dependencies, which require disk writes and introduce execution overhead. The result of the job is produced in the final stage (R).

How Spark executes a job? Implementation, scheduling, and execution. The execution of a job in Spark follows a multi-step process that involves the construction of the DAG,

its transformation into a physical execution plan, and the actual execution on the cluster. The process is:

1. **RDD creation and DAG construction**: each transformation applied to a dataset (e.g. `map`, `filter`, `groupBy`) results in the creation of a new RDD. These RDDs are linked by their dependencies, forming the DAG (the sequence of logical operations to be performed).

2. **DAG scheduling and stage division**: the DAG scheduler analyzes the graph and splits it into a sequence of stages, each composed of a set of transformations that can be executed without data shuffling (narrow dependencies). Each stage is the translated into a set of tasks, one for each data partition. These tasks are the smallest unit of execution in Spark.

3. **Task scheduling and resource allocation**: the task scheduler takes the generated tasks and submits them to the cluster manager (YARN, Kubernetes), which is responsible for allocating resources - such as executors and memory - across the cluster, The cluster manager selects appropriate nodes (workers) on which the tasks will be executed.

4. **Task execution on workers**: once scheduled, each worker node runs its assigned tasks using a thread pool. Within each executor:

   - The task is executed on the assigned partition;
   - The block manager handles data storage, managing memory, disk and caching of intermediate results;
   - Tasks may be pipelined to reduce overhead, and shuffled data is exchanged between nodes when necessary.

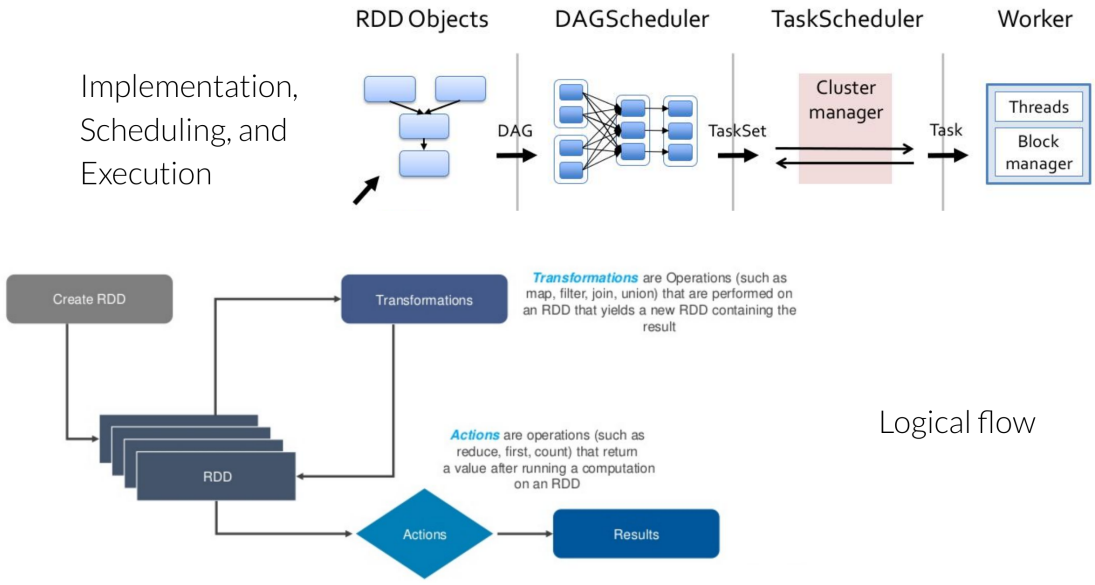


Figure 11.9: Spark execution job.

11.2.4 DataFrame

The RDD is intended as a low-level API, mostly designed primarily for semi-structured and unstructured datasets. For structured datasets, Spark offers a high-level API called **SparkSQL DataFrame**, which has similar features to the Pandas DataFrame and offers a SQL interface for queries. It is also distributed and resilient in nature as RDDs. However, underneath a DataFrame, Spark still sees an RDD.

11.3 Real time analytics

Real time processing of unbounded data flows is a common requirement across modern applications. Typical use cases include:

- **Data sources**:
  - Sensor data from physics experiments or detectors
  - Readings from IoT devices and smart infrastructure
  - Monitoring of financial or commercial transactions
  - User interaction data from websites or web applications
- **Processing objectives**:
  - Real-time data monitoring and visualization
  - Rolling or windowed aggregations over time intervals
  - Fault detection or alert generation on anomalous data
  - Online machine learning on streaming inputs.

The system must deal with both:

Data input rate: how fast data is ingested

Max affordable latency: how fast the data must be processed to remain useful

Managing real-time streaming data in distributed systems comes with significant technical challenges:

- Data arriving out of order (network delays)
- High data throughput requirements (large volume in short time)
- Reliability guarantees, such as exactly-once processing
- Load balancing across nodes to avoid bottlenecks
- and other issues like state management, fault tolerance, and time synchronization

When processing streaming data in real-time, different execution models can be adopted depending on latency constraints, throughput needs, and infrastructure capabilities. Two of the most common paradigms are:

- **One-element-at-a-time processing**. In this model, each incoming data element (event, record, message) is processed individually as soon as it arrives. The system implements a fixed pipeline of operations, always active and listening to the input sources.
  - pro **Low latency**: immediate reaction to new data
  - con **Limited throughput**: processing one record at a time can become a bottleneck
  - con **Load balancing issues**: fixed pipelines may struggle with sudden spikes in input rate

  This approach is typically implemented using event-driven architectures and is suitable for applications where immediate response is critical (e.g. anomaly detection, sensor alarms).



Figure 11.10

- **Micro-batching**. This model offers a compromise between latency and throughput. Instead of processing each data element individually, the system groups events into small batches over short, fixed time intervals (e.g. every 500 ms) and then processes each batch using parallel and distributed tasks.
  - con **Slightly higher latency**: data must wait until the micro-batch window closes
  - pro **Higher maximum throughput**: batch processing is more efficient on distributed infrastructure
  - pro **Better resource utilization**: easier to balance workload dynamically across the cluster

  This is the model adopted by Spark Structured Streaming, which processes streams as a series of small batch jobs under the hood.

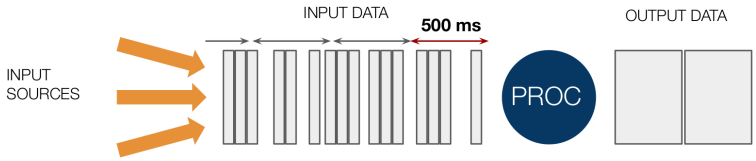


Figure 11.11

11.3.1 Real time analytics in Spark

Real-time data processing in Spark can be performed using both low- and high-level APIs:

- Spark Streaming (now deprecated)
- Spark Structured Streaming

In both cases, Spark defines two key abstractions to represent the input and output of the streaming pipeline:

- **Source** → an interface for connecting to streaming systems such as Kafka, Flume, Twitter, a TCP socket,...
- **Sink** → an abstraction for exporting the processed steam to external systems, such as to Kafka, files, DB,...

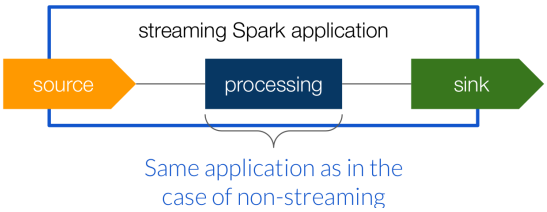


Figure 11.12

11.3.2 Real time analytics in Spark - RDD

Spark Streaming was the original API for real-time analytics based on the low-level RDD. It uses the **DStream API**, in which the streaming data is divided into small, fixed-size chunks (micro-batches), each represented as RDDs.



Figure 11.13

Key characteristics of the DStream API:

- Input data is discretized into a DStream
- A DStream element can be created from external sources or transformations on other DStreams
- DStreams can be conceptually seen as a sequence of RDDs, each representing a batch over a small time window

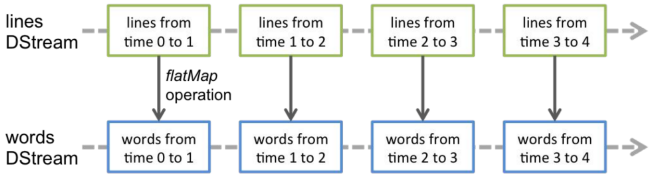


Figure 11.14

Although powerful, Spark Streaming based on RDDs is no longer actively supported and has been largely replaced by Spark Structured Streaming.

11.3.3 Real time analytics in Spark - DataFrame

Spark Structured Streaming is the recommended API for real-time analytics. Built for the high-level structured DataFrame, it embeds the streaming functionalities in the Spark SQL API and offers a more declarative approach to stream processing.

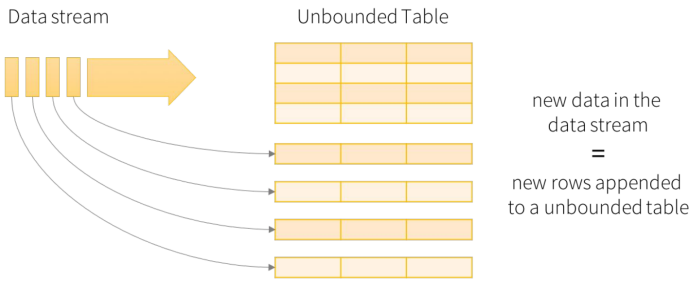


Figure 11.15

Spark Structured Streaming API blurs the line between micro-batching / OEaaT processing:

- Input data must follow a well-defined schema
- New records can be interpreted as rows continuously appended to an unbounded table
- Transformations are expressed as queries over this evolving table

Chapter 12

Dask

12.1 Dask introduction

Dask is an open source library for parallel and distributed computing in Python, designed to perfectly scale typical Python workflows to multi-core and multi-node environments.

It was developed in 2015 to fill crucial gaps in the python ecosystem: although Python includes a rich set of analytical tools (e.g. NumPy, Pandas, SciPy, Sklearn, Jupyter, Matplotlib, etc.), most of these libraries are inherently single-core and operate in-memory, which limits their performance on large datasets or computationally intensive tasks.

Dask addresses these limitations by keeping two main functionalities in mind:

1. A powerful and efficient task scheduling system that allows efficient execution of parallel computations on single machines, clusters, or HPC environments. It handles the coordination of tasks, keeps track of tasks dependencies, and assigns work across multiple machines using dynamic DAGs.

2. High-level collections that mimic familiar APIs (e.g. NumPy arrays, Pandas DataFrames, Python lists), but are internally split into many small blocks which are processed in parallel. These collections allow users to scale computations beyond memory constraints, enabling the analysis of datasets that are larger than RAM.

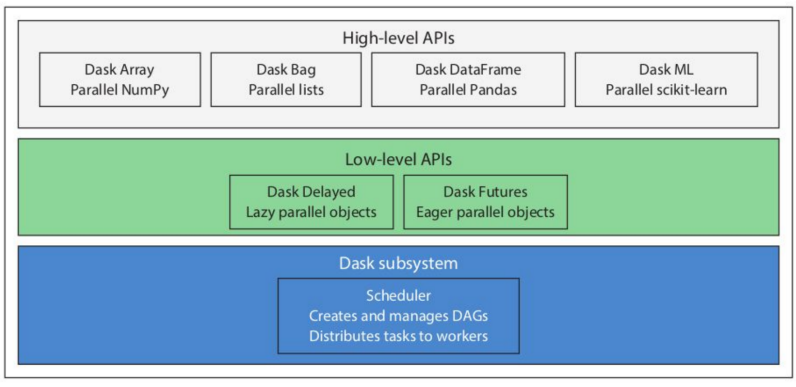


Figure 12.1: Dask architecture: layered stack from high-level collections to low-level execution.

12.2 High-level API

Dask provides high-level distributed data collections that mimic Python lists, NumPy arrays, Pandas DataFrames (see top layer in Figure 12.1). The main idea behind the high-level data collection APIs is that Dask extends their plain-python equivalent for use on parallel and distributed systems.

- A **Dask DataFrame** is made up of (several) Pandas DataFrames
- A **Dask Array** is made up of (several) NumPy Arrays
- ...

Each smaller underlying object (a **chunk**, or a **partition**) can be processed independently. These chunks may be distributed across different machines in a cluster, or handled sequentially on a single node when working locally.

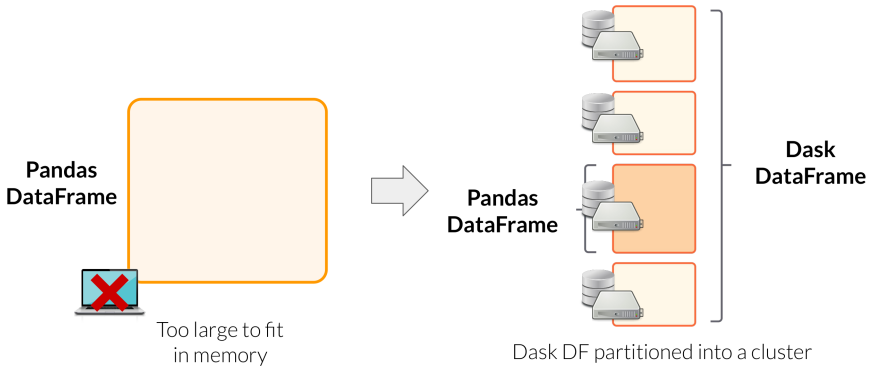


Figure 12.2: A Dask DataFrame is composed of multiple Pandas DataFrames distributed across chunks.

12.3 Low-level API

Dask provides fine-grained control for the parallel and distributed execution of any Python code [1], regardless of the usage of the high-level APIs (see central layer of Figure 12.1).

Any Python instruction or instruction can be executed through Dask either in a lazy or eager manner:

- **Lazy execution**: the code is not executed immediately. Instead, Dask builds a graph of the operations to be performed and optimizes their execution. This approach minimizes overhead and maximizes resource efficiency. For instance:

  ```
  res = delayed(my_python_function)(some_data)
  ```

  This line does not execute the function, but create a task object (`res`) that will be computed later. Actual execution starts when explicitly triggered with `res.compute()`. At that point, Dask runs the computation in parallel across available resources.

[1] Dask's low-level APIs allow the distribution and scheduling of plain Python code across threads or nodes... but it's important to realize that performance benefits are gained only if the code is written to be efficiently parallelizable

- **Eager execution**: the function is submitted and executed immediately over the cluster. This mode is particularly useful for tasks where results are needed right away or when the computation is event-driven. For example:

  ```
  p = Client.submit(my_python_function, some_data)
  ```

  The task is immediately sent to the cluster and `p` becomes a future or a placeholder for the result. Once the task is completed, the result can be retrieved using `Client.gather(p)`.

These two paradigms allow Dask to support a wide range of computational workflows:

- Simple parallel tasks (e.g. SPMD)
- Segments of complex workflows where specific functions benefit from distributed computation
- Flexible orchestration that doesn't rely on the classic MapReduce paradigm.

12.4 DAG scheduler

At its core level, Dask is a Distributed tASK scheduler, designed to efficiently run Python code across a wide range of parallel and distributed computing environments. The tasks execution is optimized via the use **Dask DAG scheduler** (bottom layer in Figure 12.1).

Dask provides two main scheduling modes:

- **Single-machine scheduler**: a lightweight scheduler that runs tasks on threads or processes within a single machine. It is simple to use and suitable for development or small workloads, but it does not scale across machines.
- **Distributed scheduler**: a more advanced and scalable scheduler that can run on multiple machines. It supports deployment on custom clusters, cloud platforms, or traditional batch systems such as SLURM or PBS. It enables distributed memory management, task monitoring, fault tolerance, and interactive dashboards.

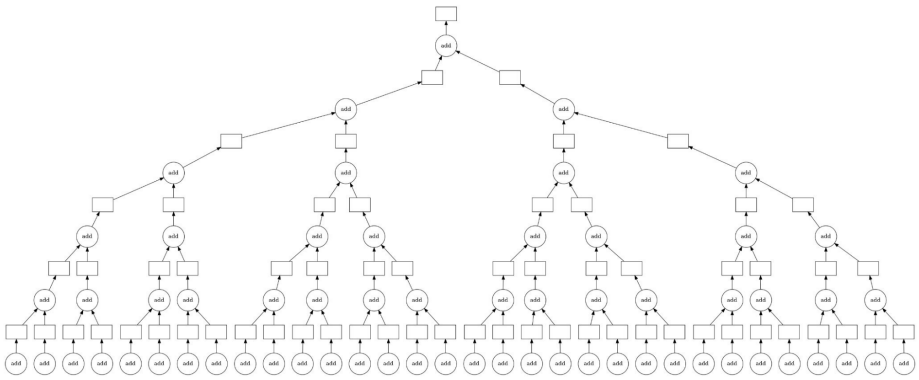


Figure 12.3

Once a task is defined using a low-level API (e.g. `dask.delayed`), Dask builds a task graph where each node represents a computational task and edges represent dependencies between them (Figure 12.3). This phase occurs before any actual computation and allows the scheduler to optimize execution.

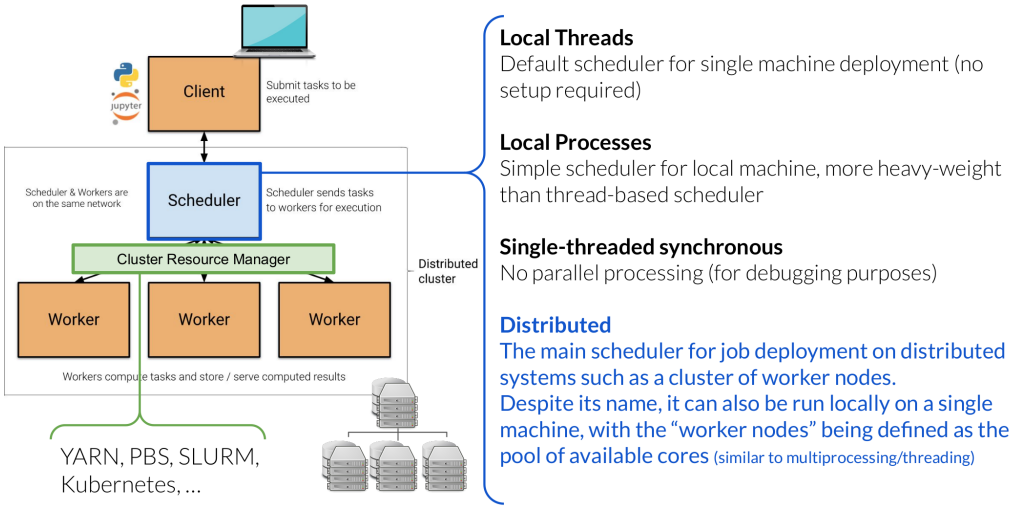


Figure 12.4: The Dask scheduler coordinates task execution either locally (on threads/processes) or across a distributed system via a resource manager.

12.5 Dask workflow

The overall Dask architecture follows a three-step logic:

1. **Collections**: high- or low-level APIs such as Dask Array, Dask DataFrame, Bag, Delayed, or Futures are used to define computations. These collections do not execute anything immediately.

2. **Task Graph**: internally, each computation is translated into a task graph—a set of operations with defined dependencies.

3. **Schedulers**: the graph is submitted to a scheduler, which coordinates and executes the tasks either on a single machine or across a cluster.

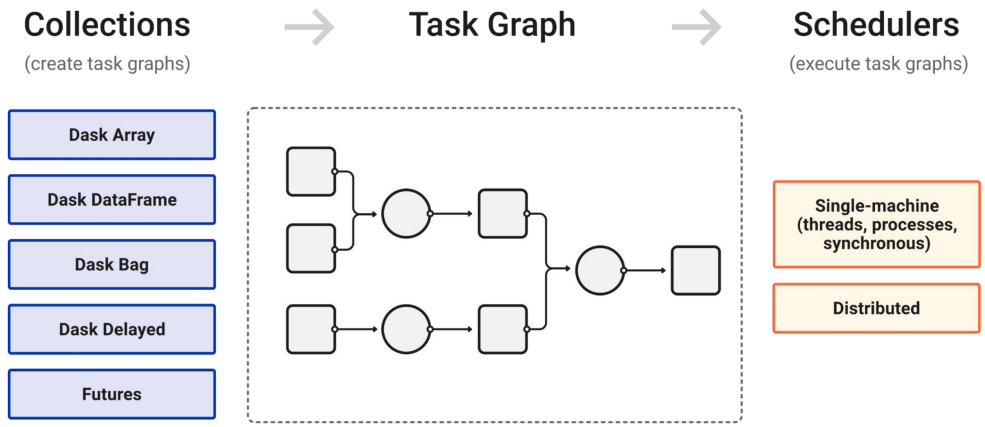


Figure 12.5: Dask workflow.

12.6 Dask vs Spark

Despite some important differences, both Dask and Spark can be used to run the same applications.

- Spark is adopted vastly in the industry, due to its all-in-one solution framework and the support of SQL.
- Dask popularity is steadily rising for scientific and data-science applications, based on the Python ecosystem.

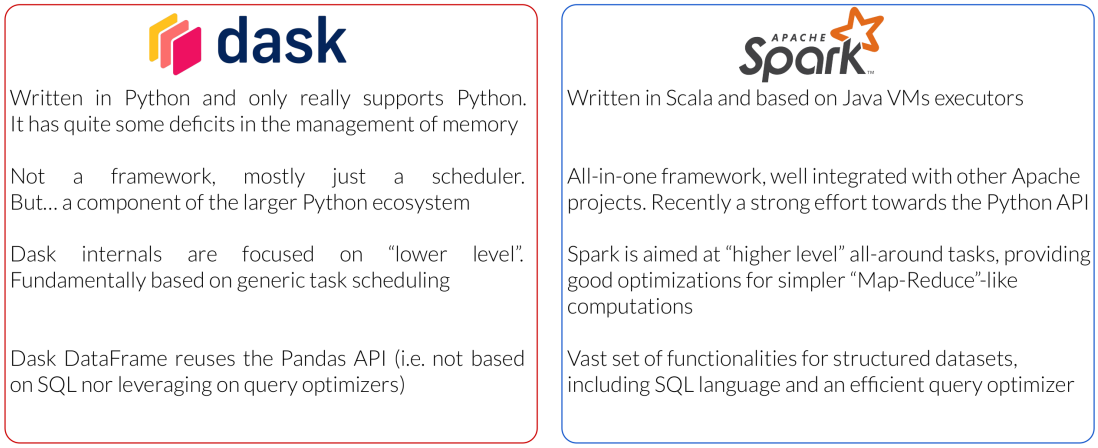


Figure 12.6